

研究平台 - 开放 API 接口文档

版本: 1.10.0

Base URL: `https://search-alpha.metastar-health.com/prod-api`

更新日期: 2026-07-16

v1.10.0 变更摘要 (向后兼容) :

- **公司源列表查询:** 新增 `GET /api/source-companies`, 查询平台监控中的公司源列表, 仅返回已启用公司。支持 `name` 按 `displayName` 或 `companyKey` 大小写不敏感模糊匹配 (中英混合名称通用; 中文需 URL 编码), 使用统一分页结构 `list / total / pageNum / pageSize / totalPages / hasNext`。每项返回 `companyKey`、`displayName`、`officialWebsite`。
- **公司源内容查询:** 新增 `GET /api/source-companies/{companyKey}/contents`, 按公司查询已采集的资讯 (news) / 演示文稿 (presentation) 条目, 按 `sortAt` (发布时间优先、回退到发现时间) 倒序。支持 `contentType` 过滤、`[startTime, endTime]` 时间范围 (作用于 `sortAt`, 闭区间)、游标翻页 (`cursor / nextCursor / hasMore`)。每条返回 `companyKey`、`contentType`、`title`、`url`、`publishedAt`、`discoveredAt`、`sortAt` 七个字段。

v1.9.1 变更摘要 (向后兼容) :

- **靶点同源数据查询:** 新增 `POST /api/homologues/search`, 支持按一个或多个靶点 `gene symbol` 查询 OpenTargets 同源关系数据。 `targets` 按大小写不敏感方式匹配 `approvedSymbol`, 返回 `approvedSymbol`、`speciesName`、`homologyType`、`targetGeneSymbol`、`queryPercentageIdentity`、`targetPercentageIdentity`、`isHighConfidence`, 并使用统一分页结构 `list / total / pageNum / pageSize / totalPages / hasNext`。

v1.9.0 变更摘要 (向后兼容) :

- **文献全文存在性查询:** 新增 `POST /api/literature-fulltext/lookup`, 支持通过 `pmid` 或 `doi` 查询平台是否存在 PubMed 全文 chunks。服务端先在平台文献摘要库做精确预检; 如未命中, 直接返回 `chunks: []`; 如命中, 再用解析出的 PMID 查询平台全文库并返回全文 chunks。默认返回 100 个 chunks, 单次上限 1000。
- **文献临床前数据检索:** `POST /api/preclinical-literature/search` 调整为文档级分页查询, 使用 `page + pageSize` 翻页, 默认每页 50 条、单页最多 100 条。返回结果按文档对象分页, 不再展开 `model_info / study`; 模型、实验和测量指标等详情保留在原始 `model_info` 中。

v1.8.1 变更摘要 (向后兼容) :

- **靶点助手提交接口:** `POST /api/target-assistant/submit` 新增 `generateReport` 参数, 默认 `false`。默认模式只执行靶点验证和数据采集/筛选, 任务完成后可直接调用 RAG 接口获取上下文, 可自定义 pipeline 生成报告; 如需平台继续生成 HTML/PDF 报告, 请显式传 `generateReport: true`。
- **靶点助手 RAG 上下文检索:** `POST /api/target-assistant/rag/search` 的 `relatedData[]` 新增全文片段挂载说明, 文献型模块可能返回 `fulltextAttachment` 与 `attachedFulltextChunks`, 用于外部系统做证据追溯和二次 RAG 编排。
- **靶点助手 RAG 全文 chunk 开关:** `POST /api/target-assistant/rag/search` 新增 `includeAllFulltextChunks` 参数, 默认 `false`。开启后, 文献型 `relatedData[]` 会返回对应 PMID 的全部可用全文 chunks, 不再由服务端做 chunk 相关性排序和筛选, 便于调用方自行排序。

v1.8.0 变更摘要 (向后兼容) :

- 新增 15 个独立检索接口, 覆盖专利临床前、专利核心库、临床试验注册/结果/全域、药物交易、靶点竞争、医药资讯、医学文献、企业动态、专利综合、实时网页、靶点综述、会议洞察、公司演示文稿检索。
- 15 个新增接口均支持同步 `execute`; 长耗时场景可使用异步 `submit + tasks/:taskId` 轮询。
- 平台生成的 appKey 认证通过后可调用本版本新增接口; 如某个接口临时不可用, 会返回统一错误响应。
- `search_drug`、`search_paper`、`search_preclinical_combine` 已有独立/存量接口, 本版本不重复开放。

- 错误响应补充字符串 `code`，包括 `tool_disabled`、`invalid_tool_input`、`max_items_exceeded`、`input_too_large`、`concurrency_limited`、`open_api_queue_full`、`open_api_app_pending_full`、`tool_submission_failed` 等。

v1.7.0 变更摘要（向后兼容）：

- **靶点助手**：新增 `POST /api/target-assistant/rag/search`，支持按 `taskId + question` 检索已完成靶点助手任务的相关模块数据和报告 chunks，适合外部系统自行做 RAG 编排或二次分析。
- **靶点助手**：`GET /api/target-assistant/result/:taskId` 完成态新增 `bulletJsonUrl` 和 `referencesCsvUrl`，分别用于下载拆解到 slide/bullet point 的 JSON 文件和报告参考文献 CSV 文件。两个文件与报告文件一样上传 OSS，接口仅返回 URL。
- **文献临床前数据检索**：新增 `POST /api/preclinical-literature/search`，支持按靶点、疾病、公司检索文献来源临床前 tool 数据；请求体兼容自动补全接口返回的 `name` / `aliases` 实体对象，响应采用 `data.status` / `data.progress` / `data.result` 包装结构。

v1.6.0 变更摘要（向后兼容）：

- **靶点助手**：新增 `POST /api/target-assistant/report/qa`，支持对已完成靶点报告进行 QA 问答；服务端默认使用 `full_task` 范围，对报告内容和已建立索引的数据统一 RAG 召回，接口参数和响应字段保持精简。

v1.5.1 变更摘要（向后兼容）：

- **接口 1 / 接口 3 (`image_info[]`)** 新增字段：`is_synthesized`（是否服务端合成的 Figure 整图）、`figure_number`（Figure 编号 "1" / "2" / ...）、`panel_count`（合成 Figure 由多少子 panel 拼合）。
- **接口 1 / 接口 3 (`image_info[]`)** 行为变更：mineru 3.x panel 级零散小图（caption 仅 "A" / "B" / "C"）服务端统一丢弃，`image_info` 仅保留完整 Figure 大图 + 完整 Table + 第 0 页 graphical abstract。单篇 PDF 项数从约 30+ 降到约 10 项（行为与 v1.4.x 对齐）。合成的 Figure 整图 URL 形如 `images/figure_N_full.png`。
- **接口 3 (`fulltext`)** 行为变更：ASM 期刊系列（mBio / mSphere / Microbiology Spectrum / J Bacteriol 等）的 inline 粘连子标题自动拆分。`RESULTS` / `METHODS` 下原本一锅的内容会按子标题展开为 `RESULTS-Bacterial growth conditions` 等多个子节。Frontiers / Cell / Nature / Elsevier 等已有独立子标题的期刊完全不受影响。
- **接口 3 (`models[].img_url`)** 可靠性提升：图片匹配从依赖单一 LLM 改为“规则匹配 + Claude Sonnet 4 兜底 + 指数退避”三层防护，命中率从约 70%（限时时跌为 0）提升到 100%。客户端字段格式不变。
- **接口 3 处理时间优化**：单篇 PDF 处理从约 5 分钟 → 约 2 分钟（视觉子图识别已关闭）。
- **全局文档规范化**：所有接口顶部新增统一的“接口元信息块”（接口类型 / Content-Type / 典型耗时 / 超时 / 轮询间隔 / 推荐并发），便于客户精确控制调用策略。

v1.5.0 变更摘要（向后兼容）：

- **OCR**：`_img_and_table.json` 中 `type` 取值统一为 `image` / `table`（mineru 3.x 内部新增的 `chart` 子类已在服务端映射回 `image`，老客户端无需调整）。ZIP 内新增 `*_content_list_v2.json`（详细布局信息，可选读取）。
- **文献数据提取**：响应 `result` 顶层新增 `paper_title` / `filename`；`models[]` 内新增 `base_characteristics`、`experimental_studies`、`img_url`；`image_info[]` 内新增 `figure_id`、`figure_caption`、`sub_labels`、`sub_caption`、`is_anchor`、`vision_used`、`is_missing`、`is_recovered`、`bbox`、`table_footnote` 等字段。所有新增字段均为附加（additive），老客户端继续使用原字段无需修改。

目录

- [认证方式](#)
- [统一响应格式](#)
- [异步任务模式](#)
- [接口类型速查表](#)
- [接口列表](#)

- [1. OCR 智能识别与切片服务](#)
- [2. 临床前信息提取服务](#)
- [3. 文献数据提取服务](#)
- [4. 药物关联论文检索](#)
- [5. 实体自动补全](#)
- [6. 靶点调研助手](#)
- [7. 靶点快速评估](#)
- [8. 药物信息查询](#)
- [9. 文献临床前数据检索](#)
- [10. HPA 靶点画像查询](#)
- [11. GBD 疾病负担数据查询](#)
- [12. 靶点同源数据查询](#)
- [13. 专利临床前数据检索](#)
- [14. 专利核心库检索](#)
- [15. 临床试验注册检索](#)
- [16. 临床试验结果检索](#)
- [17. 药物交易检索](#)
- [18. 靶点竞争分析](#)
- [19. 医药行业资讯检索](#)
- [20. 医学文献检索](#)
- [21. 企业动态检索](#)
- [22. 临床试验全域检索](#)
- [23. 专利综合检索](#)
- [24. 实时网页检索](#)
- [25. 靶点综述文献检索](#)
- [26. 会议洞察](#)
- [27. 公司演示文稿检索](#)
- [28. 文献全文存在性查询](#)
- [29. 公司源列表查询](#)
- [30. 公司源内容查询](#)
- [完整集成示例](#)

认证方式

所有接口均需通过 **App Key / App Secret** 进行身份认证。请在每个请求中携带以下 HTTP 请求头：

请求头	类型	必填	说明
<code>x-app-key</code>	string	是	应用 Key
<code>x-app-secret</code>	string	是	应用 Secret

请联系开发团队获取您的 `appKey` 和 `appSecret`。

示例：

```
GET /prod-api/api/papers/search?disease=diabetes HTTP/1.1
Host: search-alpha.metastar-health.com
x-app-key: your_app_key
x-app-secret: your_app_secret
```

认证失败响应：

```
{
  "statusCode": 401,
  "message": "Invalid app key or app secret",
  "error": "Unauthorized"
}
```

统一响应格式

成功响应

所有接口成功时返回 HTTP 200，响应体统一封装为：

```
{
  "data": { ... },
  "error": 0
}
```

字段	类型	说明
<code>data</code>	object	业务数据
<code>error</code>	number	错误码， <code>0</code> 表示成功

错误响应

```
{
  "message": "错误描述信息",
  "error": 10001
}
```

字段	类型	说明
<code>message</code>	string	可读的错误描述
<code>error</code>	number	非零错误码

HTTP 状态码

开放 API 的业务响应以响应体中的 `error` 为准。HTTP 状态码主要用于网关、认证中间件或非业务层异常的辅助判断。

状态码	说明
200	请求已被服务端处理；业务成功或失败需继续读取响应体 <code>error</code>
400	请求格式错误或参数无法被框架解析
401	认证失败（凭证无效或缺失）
403	无权限访问该工具或资源
404	路由或资源不存在
413	请求体过大
429	触发限流或队列保护
500	服务器内部错误

业务错误码

错误码	说明	解决方案
0	成功	-
100	服务器内部错误	参考 <code>message</code> 字段获取详细信息
429	触发入口限流、Open API 队列繁忙，或本地执行并发不足	降低请求频率；异步任务出现 <code>rate_limited</code> 时不要重复提交，继续轮询
1000	非法操作或无权限访问当前资源	检查工具是否被平台全局禁用或操作条件是否满足
1001	请求参数无效	检查参数格式、必填项、单次数量上限和日期/时区格式
2001	认证参数无效	检查 <code>x-app-key</code> / <code>x-app-secret</code>
3001	资源不存在	检查 <code>taskId</code> 、 <code>toolName</code> 或资源归属 <code>appKey</code> 是否正确

新增接口字符串错误码

新增的 15 个检索接口在数字 `error` 之外会返回字符串 `code`。同步接口错误直接出现在响应顶层；异步任务失败时出现在 `data.error.code`。

Code	对应业务错误码 / 任务状态	说明	处理建议
<code>tool_not_found</code>	3001	工具名不存在或未在本版本注册。	检查接口地址中的工具名； <code>search_drug</code> / <code>search_paper</code> 请使用专用接口。
<code>tool_disabled</code>	1000	接口被平台临时全局禁用。	暂停调用该接口，等待平台恢复或联系平台确认原因。
<code>sync_not_allowed</code>	1001	保留给未来仅异步工具；当前 15 个接口均支持同步 <code>execute</code> 。	当前无需处理；如未来遇到则改用 <code>submit</code> + <code>tasks/:taskId</code> 轮询。
<code>invalid_tool_input</code>	1001	请求体不符合该接口输入 schema，或 JSON 不可序列化。	根据返回 <code>details</code> 修正字段类型、必填项和嵌套路径。
<code>max_items_exceeded</code>	1001	<code>limit</code> / <code>maxResults</code> / <code>maxArticles</code> （按具体接口定义）或批量实体数量超过接口单次上限。	降低请求数量，拆分多次调用。
<code>input_too_large</code>	1001	请求体超过 <code>maxInputBytes</code> ，默认 1 MB。	精简输入，避免一次提交大量实体。
<code>task_not_found</code>	3001	当前 appKey 下找不到指定 <code>taskId</code> 。	检查 <code>taskId</code> 是否正确，或是否使用了其他 appKey。
<code>concurrency_limited</code>	429 / <code>rate_limited</code>	平台本地执行并发暂时不足，异步任务会自动延迟重排。	不要重复提交，继续轮询任务状态。
<code>open_api_queue_full</code>	429	Open API 共享任务队列积压达到平台阈值，当前请求未创建 task、未入队。	稍后重试；如持续出现，降低提交速率或联系平台扩容。
<code>open_api_app_pending_full</code>	429	当前待处理任务数达到平台阈值，当前请求未创建 task、未入队。	等待已有任务完成后再提交，或降低并发提交量。
<code>tool_submission_failed</code>	100	异步 task 已创建但入队失败，task 会被标记为 <code>failed</code> 。	稍后重试；如持续出现，联系平台排查队列。
<code>tool_execution_failed</code>	<code>failed</code>	工具执行阶段失败。	读取 <code>message</code> ，必要时联系平台排查。

上游限流和错误传递

新增接口不对智慧芽 Synapse 做额外 QPM 预限流或排队等待。智慧芽、PubMed、实时网页检索等上游服务返回限流、超时或其他错误时，平台按失败处理：

- 同步 `execute`：直接返回统一错误响应，`message` 会包含上游错误信息。
- 异步 `submit`：任务会进入 `failed` 状态，错误信息位于 `data.error`；客户端无需重复提交同一个任务。

异步任务模式

OCR、临床前信息提取、文献数据提取、靶点助手、靶点快速评估，以及本版本新增接口的 `submit` 调用均采用 **异步提交/轮询** 模式：

```
1. POST /submit      提交文件或参数  -->  返回 { taskId }
2. GET /result/:taskId  轮询任务进度    -->  返回 { status, progress, result? }
```

任务状态生命周期：

```
pending --> processing --> completed
                        \-> failed
```

状态	说明	进度范围
pending	任务已创建，等待处理	0%
processing	任务处理中	1% - 99%
completed	任务处理完成	100%
failed	任务处理失败	-

建议轮询间隔： 5 - 10 秒

接口类型速查表

客户端集成时请根据接口类型选择合适的调用方式：**同步接口**直接拉取结果；**异步接口**需先提交、再轮询。

#	接口	类型	典型耗时	超时	推荐并发	依赖 LLM
1	OCR 智能识别与切片	异步	2 - 5 分钟	10 分钟	10 个	否
2	临床前信息提取	异步	1 - 3 分钟	5 分钟	10 个	是
3	文献数据提取	异步	约 2 分钟 (v1.5.1 优化)	30 分钟	10 个	是 (兜底)
4	论文搜索	同步	1 - 3 秒	30 秒	10 个	否
5	实体自动补全	同步	100 - 500 ms	10 秒	50 个	否
6	靶点助手	异步 + 同步扩展	数据采集 10 - 30 分钟；报告生成 10 - 50 分钟；QA 1 - 360 秒；RAG 1 - 10 分钟	60 分钟	3 个	是
7	靶点快速评估	异步	1 - 5 分钟/靶点	30 分钟	5 个	是

8	药物搜索	同步	1 - 3 秒	30 秒	10 个	否
9	文献临床前数据检索	同步	1 - 3 秒	30 秒	10 个	否
10	HPA 靶点画像查询	同步	200 - 800 ms	10 秒	20 个	否
11	GBD 疾病负担数据查询	同步	1 - 3 秒	30 秒	10 个	否
12	靶点同源数据查询	同步	200 ms - 2 秒	30 秒	20 个	否
13	专利临床前数据检索	同步 + 异步	1 - 15 秒	30 秒	5 个	否
14	专利核心库检索	同步 + 异步	5 - 30 秒	60 秒	5 个；无智慧芽 QPM 预限流	否
15	临床试验注册检索	同步 + 异步	5 - 30 秒	60 秒	5 个；无智慧芽 QPM 预限流	否
16	临床试验结果检索	同步 + 异步	5 - 60 秒	90 秒	5 个；无智慧芽 QPM 预限流	否
17	药物交易检索	同步 + 异步	5 - 30 秒	60 秒	5 个；无智慧芽 QPM 预限流	否
18	靶点竞争分析	同步 + 异步	5 - 60 秒	60 秒	5 个；无智慧芽 QPM 预限流	否
19	医药行业资讯检索	同步 + 异步	5 - 30 秒	60 秒	5 个；无智慧芽 QPM 预限流	否
20	医学文献检索	同步 + 异步	5 - 30 秒	60 秒	5 个	否
21	企业动态检索	同步 + 异步	1 - 15 秒	30 秒	5 个	否
22	临床试验全域检索	同步 + 异步	1 - 15 秒	30 秒	5 个	否
23	专利综合检索	同步 + 异步	1 - 15 秒	30 秒	5 个	否
24	实时网页检索	同步 + 异步	5 - 30 秒	60 秒	5 个	否
25	靶点综述文献检索	同步 + 异步	1 - 15 秒	30 秒	5 个	否
26	会议洞察	同步 + 异步	1 - 15 秒	30 秒	5 个	否
27	公司演示文稿检索	同步 + 异步	1 - 15 秒	30 秒	5 个	否

28	文献全文存在性查询	同步	200 ms - 3 秒	30 秒	20 个	否
29	公司源列表查询	同步	100 - 500 ms	10 秒	20 个	否
30	公司源内容查询	同步	200 ms - 2 秒	30 秒	20 个	否

各接口顶部均有完整的“接口元信息块”（Content-Type、轮询间隔、结果保留期限等），调用前请先阅读。

接口列表

1. OCR 智能识别与切片服务

提供基于先进 OCR 技术的 PDF 文档智能识别与自动切片功能，支持大文档分块处理，提取文本、表格和图像内容。

接口类型：异步任务（提交 + 轮询）
Content-Type: multipart/form-data
典型耗时：2 - 5 分钟（取决于 PDF 复杂度与页数）
超时上限：10 分钟
建议轮询间隔：5 - 10 秒
并发限制：10 个/秒
结果保留：24 小时（OSS URL 持续可用 7 天）

1.1 提交 OCR 任务

```
POST /api/ocr/submit
```

Content-Type: multipart/form-data

请求参数：

参数名	类型	必填	说明
file	File	是	待识别的 PDF 文件，最大 50 MB

JavaScript 示例：

```
const formData = new FormData();
formData.append("file", pdfFile);

const response = await fetch(
  "https://search-alpha.metastar-health.com/prod-api/api/ocr/submit",
  {
    method: "POST",
    headers: {
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
    body: formData,
  },
);
```

```
const result = await response.json();
// result.data.taskId
```

Python 示例:

```
import requests

url = "https://search-alpha.metastar-health.com/prod-api/api/ocr/submit"
headers = {
    "x-app-key": "your_app_key",
    "x-app-secret": "your_app_secret",
}
files = {"file": ("document.pdf", open("document.pdf", "rb"), "application/pdf") }

response = requests.post(url, headers=headers, files=files)
result = response.json()
task_id = result["data"]["taskId"]
```

成功响应:

```
{
  "data": {
    "taskId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "error": 0
}
```

错误响应 (文件类型不合法) :

```
{
  "message": "Only PDF files are allowed",
  "error": 10001
}
```

1.2 查询 OCR 结果

```
GET /api/ocr/result/:taskId
```

路径参数:

参数名	类型	必填	说明
taskId	string	是	提交任务时返回的任务 ID

JavaScript 示例:

```
const response = await fetch(
  `https://search-alpha.metastar-health.com/prod-api/api/ocr/result/${taskId}`,
  {
    headers: {
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
  },
);
const result = await response.json();
```

Python 示例:

```
response = requests.get(
    f"https://search-alpha.metastar-health.com/prod-api/api/ocr/result/{task_id}",
    headers=headers,
)
result = response.json()
```

响应示例（处理中）：

```
{
  "data": {
    "taskId": "550e8400-e29b-41d4-a716-446655440000",
    "status": "processing",
    "progress": 45,
    "createdAt": "2026-04-04T10:00:00.000Z",
    "updatedAt": "2026-04-04T10:02:30.000Z"
  },
  "error": 0
}
```

响应示例（处理完成）：

```
{
  "data": {
    "taskId": "550e8400-e29b-41d4-a716-446655440000",
    "status": "completed",
    "progress": 100,
    "result": {
      "downloadUrl": "https://oss.example.com/open-api/ocr/abc123.zip",
      "filename": "document_result.zip",
      "size": 1643912
    },
    "createdAt": "2026-04-04T10:00:00.000Z",
    "updatedAt": "2026-04-04T10:05:00.000Z"
  },
  "error": 0
}
```

响应示例（处理失败）：

```
{
  "data": {
    "taskId": "550e8400-e29b-41d4-a716-446655440000",
    "status": "failed",
    "progress": 100,
    "error": "PDF 文件损坏或无法处理",
    "createdAt": "2026-04-04T10:00:00.000Z",
    "updatedAt": "2026-04-04T10:01:00.000Z"
  },
  "error": 0
}
```

结果字段说明：

字段	类型	说明
result.downloadUrl	string	ZIP 文件下载地址
result.filename	string	文件名（如 document_result.zip）
result.size	number	ZIP 文件大小（字节）

ZIP 文件内容结构：

```
{filename}_result.zip
├── {filename}_result/
│   ├── {filename}/
│   │   ├── {filename}.md                # 完整文档 Markdown 格式
│   │   ├── slices.json                 # 结构化文本切片数据
│   │   ├── {filename}_img_and_table.json # 图片和表格元数据
│   │   └── images/                     # 提取的图片文件夹
│   │       ├── {hash1}.jpg
│   │       └── ...
```

文件名	格式	描述
*.md	Markdown	完整文档的结构化文本内容
slices.json	JSON	按指定长度切片的文本数据，便于后续处理
*_img_and_table.json	JSON	图片和表格的位置、描述等元数据信息（字段同接口 3 image_info[]）
*_content_list_v2.json	JSON	mineru 3.x 详细布局信息（含 bbox / 子区块等，进阶用途，v1.5.0 起新增）
images/*.jpg	JPEG	从 PDF 中提取的图片文件（panel 级）
images/figure_N_full.png 🌟	PNG	v1.5.1 起新增：服务端按 caption 锚定合成的 Figure N 完整大图，与 image_info[].is_synthesized=true 一一对应

注意事项：

- 仅支持 PDF 格式文件
- 最大文件大小：50 MB
- 处理超时时间：10 分钟

- 任务完成后通过 `downloadUrl` 直接下载 ZIP 文件
- **v1.5.1 起** `*_img_and_table.json` 项数变化提示: mineru 3.x panel 级零散小图 (caption 仅 `"A"` / `"B"` / `"C"`) 服务端统一丢弃, 单篇 PDF 图项数从约 30+ 降到约 10 项。客户端按 `figure_number` 渲染整图即可, 与 v1.4.x 行为对齐。详见接口 3 `image_info[]` 字段说明

2. 临床前信息提取服务

提供科研文本内容智能分析功能, 专门用于从科研文本中自动识别和提取动物模型、细胞模型等临床前相关信息。

接口类型: 异步任务 (提交 + 轮询)
Content-Type: `multipart/form-data`
典型耗时: 1 - 3 分钟 (取决于文本长度与模型数)
超时上限: 5 分钟
建议轮询间隔: 5 - 10 秒
并发限制: 10 个/秒
结果保留: 24 小时

2.1 提交文本提取任务

POST `/api/text-extraction/submit`

Content-Type: `multipart/form-data`

请求参数:

参数名	类型	必填	说明
<code>file</code>	File	是	待分析的纯文本文件 (<code>.txt</code> , UTF-8 编码, 最大 50 MB)

JavaScript 示例:

```
const formData = new FormData();
formData.append("file", txtFile);

const response = await fetch(
  "https://search-alpha.metastar-health.com/prod-api/api/text-extraction/submit",
  {
    method: "POST",
    headers: {
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
    body: formData,
  },
);
const result = await response.json();
```

Python 示例:

```
url = "https://search-alpha.metastar-health.com/prod-api/api/text-extraction/submit"
files = {"file": ("sample.txt", open("sample.txt", "rb"), "text/plain")}

response = requests.post(url, headers=headers, files=files)
task_id = response.json()["data"]["taskId"]
```

成功响应：

```
{
  "data": {
    "taskId": "660f9500-f39c-52e5-b827-557766550000"
  },
  "error": 0
}
```

2.2 查询文本提取结果

```
GET /api/text-extraction/result/:taskId
```

路径参数：

参数名	类型	必填	说明
taskId	string	是	提交任务时返回的任务 ID

响应示例（处理完成）：

```
{
  "data": {
    "taskId": "660f9500-f39c-52e5-b827-557766550000",
    "status": "completed",
    "progress": 100,
    "result": [
      {
        "success": true,
        "pmid": "N/A",
        "title": "文件名",
        "model_names": [
          {
            "model_id": "model_001",
            "model_type": "animal_model",
            "model_name": "C57BL/6 mice",
            "brief_description": "糖尿病研究小鼠模型"
          }
        ]
      },
      {
        "model_id": "model_001",
        "model_name": "C57BL/6 mice",
        "model_type": "animal_model",
        "model_purpose": "药效评价",
        "brief_description": "详细模型描述",
        "extracted_fields": {
          "species": "小鼠",
          "strain": "C57BL/6",
          "age": "8-10周",
          "sex": "雄性",
          "sample_size": "n=10",
          "treatment_duration": "4周",
          "dosage": "10 mg/kg",
          "administration_route": "腹腔注射"
        }
      }
    ]
  },
  "error": 0
}
```

```
        "quality_check": {
            "corrections": [],
            "completeness_score": 0.85
        }
    },
    "processing_summary": {
        "total_models": 1,
        "animal_models": 1,
        "cell_models": 0
    }
},
"createdAt": "2026-04-04T10:00:00.000Z",
"updatedAt": "2026-04-04T10:03:00.000Z"
},
"error": 0
}
```

模型对象字段说明：

字段	类型	说明
model_id	string	模型唯一标识符
model_type	string	模型类型： animal_model （动物模型）或 cell_model （细胞模型）
model_name	string	模型名称
model_purpose	string	模型用途分类
brief_description	string	模型简要描述
extracted_fields	object	详细的模型参数信息（物种、品系、年龄、给药方案等）
quality_check	object	质量检查和修正信息

注意事项：

- 仅支持 UTF-8 编码的纯文本文件（.txt）
- 最大文件大小：50 MB
- 处理超时时间：5 分钟
- 输入文本应包含完整的实验方法描述，以获得最佳提取效果

3. 文献数据提取服务

PDF 文献智能解析服务，专门用于从学术论文中自动提取临床前研究数据，包括全文结构化内容、实验模型信息和图表数据。

接口类型： 异步任务（提交 + 轮询）

Content-Type: multipart/form-data

典型耗时： 约 2 分钟（v1.5.1 起，原生 PDF；v1.5.0 及之前约 5 分钟）

超时上限： 30 分钟

建议轮询间隔： 10 - 15 秒

并发限制： 10 个/秒（同账户）

结果保留： 7 天（OSS 图片资源 30 天）

依赖外部 LLM： 是（章节抽取 + 图片匹配兜底走 Claude Sonnet 4，规则匹配命中时跳过 LLM）

3.1 提交文献解析任务

POST /api/literature-process/submit

Content-Type: multipart/form-data

请求参数:

参数名	类型	必填	说明
file	File	是	PDF 文献文件，最大 50 MB

JavaScript 示例:

```
const formData = new FormData();
formData.append("file", pdfFile);

const response = await fetch(
  "https://search-alpha.metastar-health.com/prod-api/api/literature-process/submit",
  {
    method: "POST",
    headers: {
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
    body: formData,
  },
);
const result = await response.json();
```

Python 示例:

```
url = "https://search-alpha.metastar-health.com/prod-api/api/literature-process/submit"
files = {"file": ("paper.pdf", open("paper.pdf", "rb"), "application/pdf")}

response = requests.post(url, headers=headers, files=files)
task_id = response.json()["data"]["taskId"]
```

成功响应:

```
{
  "data": {
    "taskId": "770a0600-a40d-63f6-c938-668877660000"
  },
  "error": 0
}
```

3.2 查询文献解析结果

GET /api/literature-process/result/:taskId

路径参数:

参数名	类型	必填	说明
taskId	string	是	提交任务时返回的任务 ID

响应示例（处理完成）：

```
{
  "data": {
    "taskId": "770a0600-a40d-63f6-c938-668877660000",
    "status": "completed",
    "progress": 100,
    "result": {
      "PMID": "12345678",
      "filename": "paper.pdf",
      "paper_title": "CTP Synthase 1 Is a Novel Therapeutic Target in Lymphoma",
      "fulltext": {
        "title": ["论文标题"],
        "ABSTRACT": ["摘要内容..."],
        "INTRODUCTION": ["引言内容..."],
        "MATERIALS AND METHODS": ["方法部分总览..."],
        "MATERIALS AND METHODS-In silico modeling": ["子章节内容..."],
        "RESULTS": ["结果部分总览..."],
        "RESULTS-Identification of selective inhibitors": ["子章节内容..."],
        "DISCUSSION": ["讨论部分内容..."],
        "REFERENCES": ["参考文献..."]
      },
    },
    "models": [
      {
        "model_id": "model_1",
        "model_name": "C57BL/6 小鼠糖尿病模型",
        "model_type": "动物模型",
        "model_purpose": "药效评价",
        "base_characteristics": {
          "species": "小鼠",
          "strain": "C57BL/6",
          "age": "8-10 周",
          "sex": "雄性"
        },
        "experimental_studies": [
          {
            "study_name": "STZ 诱导糖尿病模型",
            "sample_size": "n=10",
            "treatment_duration": "4 周",
            "dosage": "10 mg/kg",
            "administration_route": "腹腔注射",
            "endpoints": ["体重变化", "血糖水平", "组织病理"],
            "statistical_methods": ["t-test", "ANOVA"],
            "significance_level": "P<0.05"
          }
        ],
        "img_url": ["https://oss.example.com/.../figure_1.jpg"]
      }
    ],
    "image_info": [
      {
        "type": "image",
        "img_path": "https://oss.example.com/.../images/figure_1_full.png",
      }
    ]
  }
}
```

```

    "image_caption": [
      "Figure 1: Characterization of small molecule selective inhibitors of CTPS1"
    ],
    "image_footnote": ["*P<0.05 vs 对照组"],
    "page_idx": 3,
    "bbox": [92, 329, 929, 699],
    "figure_id": "Figure 1",
    "figure_number": "1",
    "figure_caption": "Characterization of small molecule selective inhibitors of CTPS1",
    "sub_labels": ["A", "B", "C"],
    "sub_caption": "Chemical structures of 2 selective CTPS1 inhibitors",
    "is_synthesized": true,
    "panel_count": 3,
    "is_anchor": true,
    "vision_used": false,
    "is_missing": false,
    "is_recovered": false
  },
  {
    "type": "table",
    "img_path": "https://oss.example.com/.../table_1.jpg",
    "table_caption": ["Table 1: 实验动物基本信息"],
    "table_footnote": ["IC50 = half maximal inhibitory concentration"],
    "table_body": "组别\t动物数量\t体重(g)\t年龄(周)\n对照组\t10\t25.2±2.1\t8-10\n治疗组\t10\t24.8±1.9\t8-10",
    "page_idx": 2,
    "bbox": [80, 578, 939, 686],
    "figure_id": "Table 1",
    "figure_number": "1",
    "figure_caption": "实验动物基本信息",
    "sub_labels": [],
    "sub_caption": "实验动物基本信息",
    "is_synthesized": false,
    "panel_count": 0,
    "is_anchor": false,
    "vision_used": false,
    "is_missing": false,
    "is_recovered": false
  }
]
},
"createdAt": "2026-04-04T10:00:00.000Z",
"updatedAt": "2026-04-04T10:08:00.000Z"
},
"error": 0
}
```

结果数据字段说明：

字段标记 🌟 表示 v1.5.0 起新增、🌟🌟 表示 v1.5.1 起新增（均向后兼容，老客户端可忽略）。

字段	类型	说明
PMID	string	PubMed ID
filename 🌟	string	上传 PDF 的文件名
paper_title 🌟	string	论文标题（自动从首页 / Title 章节提取）

fulltext	object	全文内容，按论文实际章节标题动态展开（key 为章节名）
fulltext.<章节名>	string[]	该章节的段落数组。章节名取自 PDF 实际结构，常见的有 title / ABSTRACT / INTRODUCTION / MATERIALS AND METHODS / RESULTS / DISCUSSION / CONCLUSION / REFERENCES，以及形如 RESULTS-Identification of selective inhibitors 的子章节
models	array	提取的实验模型信息
models[].model_id	string	模型唯一标识符
models[].model_name	string	模型名称
models[].model_type	string	模型类型（动物模型 / 细胞模型）
models[].model_purpose	string	模型用途
models[].base_characteristics ✨	object	模型基础特征（物种 / 品系 / 年龄 / 性别等）
models[].experimental_studies ✨	array	基于该模型的实验研究列表（剂量 / 疗程 / 给药途径 / 终点 / 统计方法等）
models[].img_url ✨	string[]	模型相关图表的 OSS URL 列表
models[].extracted_fields	object	旧版详细参数对象（v1.5.0 起逐步迁移到 base_characteristics + experimental_studies，仍保留以兼容老客户端）
image_info	array	提取的图表信息
image_info[].type	string	image（图片，含 chart）或 table（表格）
image_info[].img_path	string	图 / 表对应图片的 OSS URL
image_info[].page_idx	number	在 PDF 中的页码（从 0 起）
image_info[].bbox ✨	number[]	图 / 表在 PDF 页面中的边界框 [x0, y0, x1, y1]，坐标范围 [0, 1000] 归一化
image_info[].figure_id ✨	string null	所属图编号（如 "Figure 1" / "Table 2"），未识别时为 null
image_info[].figure_number ✨✨	string null	Figure 编号字符串（如 "1" / "2"），便于按编号匹配；与 figure_id 配套，未识别时为 null
image_info[].figure_caption ✨	string	完整的图 / 表标题（含编号）
image_info[].sub_labels ✨	string[]	子图标签数组（如 ["A", "B", "C"]），合成大图时为该 Figure 内全部子图标签，无子图时为 []
image_info[].sub_caption ✨	string	子图或当前对象的具体描述
image_info[].is_synthesized ✨✨	boolean	是否为服务端合成的 Figure 完整大图。true 表示该项由多个 panel 拼合（推荐客户端展示），false 表示来自 OCR 原图（Table 或 graphical abstract）
image_info[].panel_count ✨✨	number	合成 Figure 时由多少个子 panel 拼合而成；非合成图为 0

<code>image_info[].is_anchor</code> ✨	boolean	是否为该 <code>figure_id</code> 的主图（用于多子图合并展示）
<code>image_info[].vision_used</code> ✨	boolean	是否启用了多模态视觉识别生成 caption（v1.5.1 起默认关闭以加速处理）
<code>image_info[].is_missing</code> ✨	boolean	是否为占位项（图片提取失败的兜底记录）
<code>image_info[].is_recovered</code> ✨	boolean	是否经过恢复处理（针对 <code>is_missing</code> 项的二次尝试结果）
<code>image_info[].image_caption</code>	string[]	图片标题段落（仅 <code>type=image</code> ）
<code>image_info[].image_footnote</code>	string[]	图片脚注段落（仅 <code>type=image</code> ）
<code>image_info[].table_caption</code>	string[]	表格标题段落（仅 <code>type=table</code> ）
<code>image_info[].table_footnote</code> ✨	string[]	表格脚注段落（仅 <code>type=table</code> ）
<code>image_info[].table_body</code>	string	表格内容文本（TSV 格式，仅 <code>type=table</code> ）

注意事项：

- 仅支持 PDF 格式文件
- 最大文件大小：50 MB
- 处理超时时间：30 分钟
- 优先使用原生 PDF（非扫描版），可获得最佳提取效果
- 处理时间：v1.5.1 起约 **2 分钟**（v1.5.0 及之前约 5 分钟，已关闭视觉子图识别）
- **v1.5.1 `image_info` 行为说明：**
 - 仅保留完整 **Figure 大图**（`is_synthesized=true`）+ **完整 Table** + 第 0 页 graphical abstract，单篇 PDF 项数从约 30+ 降到约 10 项
 - mineru 3.x panel 级零散小图（caption 仅 "A" / "B" / "C"）服务端统一丢弃，不再下发，与 v1.4.x 行为对齐
 - 客户端按 `figure_number` 渲染整图、按 `sub_labels` 展示子图说明即可
- **v1.5.1 `fulltext` 章节拆分说明：**
 - ASM 期刊系列（mBio / mSphere / Microbiology Spectrum / J Bacteriol 等）的 inline 粘连子标题（"xxxx." 句号后接大写正文）自动拆分为独立的 `RESULTS-xxxx` / `METHODS-xxxx` 子节
 - Frontiers / Cell / Nature / Elsevier 等已有独立子标题的期刊**完全不受影响**
 - 仅在 `RESULTS` / `METHODS` / `MATERIALS AND METHODS` / `EXPERIMENTAL PROCEDURES/SECTION` 内部启用，且只在该一级章节**没有任何 ## 子标题**时触发
- **v1.5.1 `models[].img_url` 三层防护：**
 1. **本地规则匹配：**按 `studies[].study_data_figure_table` / `data_figure_table` 中的 `Figure N` / `Table N` 编号直接对应 `image_info[].figure_number`（零 LLM 调用、零费用、毫秒级）
 2. **LLM 兜底：**未命中时降级到 Claude Sonnet 4（仅在 Supplementary / Extended Data 等非标准引用时调用）
 3. **指数退避：**LLM 限流时按 5s / 15s / 45s 自动重试，最多 3 次
 - 标准期刊（fimmu / Cell / Nature 等）99% 走规则匹配，命中率从约 70% 提升到 100%

4. 药物关联论文检索

提供强大的学术论文数据查询功能，支持按疾病、药物、靶点、公司等多种条件搜索。

接口类型： 同步（实时返回）
Content-Type: application/x-www-form-urlencoded （query string）
典型耗时： 1 - 3 秒
响应超时： 30 秒
并发限制： 10 个/秒
数据源： Elasticsearch 论文索引（实时）

4.1 搜索论文

GET /api/papers/search

查询参数：

参数名	类型	必填	默认值	说明
disease	string	否*	-	疾病/适应症名称，支持分号分隔多个疾病
drug	string	否*	-	药物名称，支持分号分隔多个药物
target	string	否*	-	靶点/基因名称，支持分号分隔多个靶点
company	string	否*	-	公司名称，支持分号分隔多个公司
limit	integer	否	20	返回结果数量（1-200）

* 至少需要提供一个搜索条件（disease、drug、target、company 中的任意一个）。

搜索逻辑：

- 同一参数内多个值（分号分隔）为 "或" 关系
- 不同参数之间为 "与" 关系

JavaScript 示例：

```
const params = new URLSearchParams({
  disease: "diabetes",
  drug: "semaglutide",
  limit: "20",
});

const response = await fetch(
  `https://search-alpha.metastar-health.com/prod-api/api/papers/search?${params}`,
  {
    headers: {
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
  },
);

const result = await response.json();
```

Python 示例：

```

url = "https://search-alpha.metastar-health.com/prod-api/api/papers/search"
params = {
    "disease": "diabetes",
    "drug": "semaglutide",
    "limit": 20,
}

response = requests.get(url, headers=headers, params=params)
result = response.json()

```

成功响应:

```

{
  "data": {
    "data": [
      {
        "referenceId": "2413872",
        "referenceTitle": "Ginsenoside Rb2 Attenuates UV-B Radiation...",
        "referenceTypes": "[ 'Journal' ]",
        "pmid": 26043782,
        "pmc": "",
        "pubdate": 2015,
        "article_types": "D016428:Journal Article",
        "journal_name": "Pharmacology",
        "journal_abbreviation": "Pharmacology",
        "issn": "0031-7012",
        "doi": "10.1159/000431154",
        "if_oid": 2.9,
        "abstract": "OBJECTIVE The antiphototoaging activities of ginsenoside Rb2...",
        "keywords": "",
        "Drug Names": "ginsenoside C;ginsenoside Rb2",
        "Target Names": "nonspecified subtype",
        "Company Names": "Astellas Pharma (Originator)",
        "Disease Names": "Diabetes",
        "url": "https://dx.doi.org/10.1159/000431154",
        "source": "u"
      }
    ],
    "total": 200,
    "limit": 20,
    "query": {
      "disease": "diabetes",
      "drug": "semaglutide"
    },
    "error": 0
  },
  "error": 0
}

```

论文对象字段说明:

字段	类型	说明
referenceId	string	论文唯一标识符
referenceTitle	string	论文标题
referenceTypes	string	论文类型
pmid	number	PubMed ID
pmc	string	PMC ID
pubdate	number	发表日期（数字格式：YYYYMMDD 或 YYYY）
article_types	string	文章类型（逗号分隔）
journal_name	string	期刊名称
journal_abbreviation	string	期刊缩写
issn	string	ISSN 号
doi	string	DOI
if_oid	number	影响因子
abstract	string	摘要
keywords	string	关键词（逗号分隔）
Drug Names	string	相关药物名称（分号分隔）
Target Names	string	相关靶点名称（分号分隔）
Company Names	string	相关公司名称（分号分隔）
Disease Names	string	相关疾病名称（分号分隔）
url	string	论文链接
source	string	数据来源

常见查询示例：

```
# 单一疾病搜索
GET /api/papers/search?disease=diabetes&limit=10

# 多靶点搜索（OR 逻辑）
GET /api/papers/search?target=GIP;GLP-1&limit=50

# 多条件组合搜索（AND 逻辑）
GET /api/papers/search?disease=diabetes&drug=semaglutide&company=Novo%20Nordisk

# 多疾病搜索
GET /api/papers/search?disease=diabetes;cancer;hypertension&limit=30

# 多药物搜索
GET /api/papers/search?drug=semaglutide;metformin;insulin&limit=40
```

错误响应示例：

```
{
  "message": "至少需要提供一个搜索条件：disease, drug, target, 或 company",
  "error": 100
}
```

4.2 健康检查

GET /api/papers/health

响应:

```
{
  "data": {
    "status": "healthy",
    "timestamp": "2026-04-04T02:35:29.883Z",
    "service": "papers-search-api",
    "elasticsearch": "connected",
    "error": 0
  },
  "error": 0
}
```

注意事项:

- 单次查询最多返回 200 条结果
- API 响应时间通常在 1-3 秒内
- 建议并发请求控制在 10 个/秒以内

5. 实体自动补全

提供疾病、靶点、药物等生物医学实体的智能搜索和自动补全功能，基于 Elasticsearch 实现，支持前缀匹配、模糊匹配和别名匹配。

接口类型： 同步（实时返回）

Content-Type: application/x-www-form-urlencoded （query string）

典型耗时： 100 - 500 ms

响应超时： 10 秒

并发限制： 50 个/秒

数据源： Elasticsearch 自动补全索引

5.1 搜索实体

GET /api/ai/autocomplete/entities

查询参数:

参数名	类型	必填	默认值	取值范围	说明
query	string	是	-	-	搜索查询词，支持前缀匹配
size	integer	否	20	1-50	返回结果数量
type	string	否	-	disease、target、drug、company	实体类型过滤

JavaScript 示例:

```
const params = new URLSearchParams({
  query: "GLP",
  type: "target",
  size: "5",
});

const response = await fetch(
  `https://search-alpha.metastar-health.com/prod-api/api/ai/autocomplete/entities?${params}`,
  {
    headers: {
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
  },
);
const result = await response.json();
```

Python 示例:

```
url = "https://search-alpha.metastar-health.com/prod-api/api/ai/autocomplete/entities"
params = {"query": "GLP", "type": "target", "size": 5}

response = requests.get(url, headers=headers, params=params)
result = response.json()
```

成功响应:

```
{
  "data": {
    "suggestions": [
      {
        "name": "euchromatic histone lysine methyltransferase 1",
        "type": "target",
        "aliases": "Euchromatic histone-lysine N-methyltransferase 1;GLP1;GLP;EHMT1",
        "_score": 40.73,
        "highest_status": null,
        "highest_status_sort": null
      },
      {
        "name": "GLP-1-GIP",
        "type": "drug",
        "aliases": "GLP-1-GIP",
        "_score": 9.03,
        "highest_status": "Preclinical",
        "highest_status_sort": 1
      }
    ],
    "total": 5,
    "query": "GLP"
  },
  "error": 0
}
```

实体对象字段说明:

字段	类型	说明
name	string	实体标准名称
type	string	实体类型: disease (疾病)、target (靶点)、drug (药物)、company (公司)
aliases	string	实体别名列表 (分号分隔)
_score	number	Elasticsearch 匹配分数, 分数越高匹配度越好
highest_status	string null	药物最高开发状态 (仅药物类型有值)
highest_status_sort	number null	开发状态排序权重 (仅药物类型有值)

智能匹配策略:

- **精确匹配:** 完全匹配实体名称或别名 (权重最高, boost: 10.0)
- **前缀匹配:** 匹配以查询词开头的实体 (权重中等, boost: 3.0)
- **模糊匹配:** 容错匹配, 支持轻微拼写错误 (权重较低, boost: 1.0)

结果排序规则:

1. 类型优先级: 疾病 > 靶点 > 药物 > 公司
2. 药物状态排序: 已上市 > III 期 > II 期 > I 期 > 临床前
3. 匹配分数: 相同类型内按 _score 降序排列

注意事项:

- query 参数不能为空或只包含空格
- 查询词前的 / 符号会被自动去除
- 未指定 type 时返回所有类型的实体, 按类型优先级排序
- 响应时间通常在 100-500ms 内

6. 靶点调研助手

提供靶点研究服务, 包括: 靶点验证 -> 数据采集/筛选; 如请求参数 generateReport=true, 会继续执行报告生成 -> PDF 导出。采用异步任务模式。

接口类型: 主流程为异步任务 (提交 + 轮询); 报告 QA / RAG 检索为同步扩展接口

Content-Type: application/json

典型耗时: 数据采集模式通常 10 - 30 分钟; 启用报告生成时主流程 10 - 50 分钟 (取决于靶点复杂度与已有数据缓存命中); QA / RAG 检索通常 1 - 10 秒

超时上限: 主流程 60 分钟; QA / RAG 检索 180 秒

建议轮询间隔: 主流程 10 秒

并发限制: 3 个/秒 (同账户最大并发任务 5 个)

结果保留: 30 天 (如启用报告生成, PDF / HTML / JSON / CSV 报告产物 OSS URL 持续可用)

依赖外部 LLM: 是 (靶点验证、数据推荐与可选报告生成依赖 LLM; RAG 检索接口仅召回上下文, 不生成答案)

6.1 提交靶点研究任务

```
POST /api/target-assistant/submit
```

Content-Type: application/json

支持两种输入方式：

- 方式一：传入靶点名称字符串（`target`），系统自动进行靶点验证（自动补全 + LLM 匹配）
- 方式二：传入预校验的靶点实体（`targetEntity`），跳过验证直接开始研究

请求参数：

字段	类型	必填	默认值	说明
<code>target</code>	string	否*	-	靶点名称字符串，会自动验证
<code>targetEntity</code>	object	否*	-	预校验的靶点实体对象
<code>targetEntity.name</code>	string	是**	-	靶点名称
<code>targetEntity.aliases</code>	string[]	否	<code>[]</code>	靶点别名列表
<code>language</code>	string	否	<code>"zh-CN"</code>	输出报告语言（ <code>zh-CN</code> 或 <code>en-US</code> ）
<code>generateReport</code>	boolean	否	<code>false</code>	是否由平台生成报告和导出文件。默认 <code>false</code> ，只完成数据采集/筛选；传 <code>true</code> 时继续生成 HTML/PDF 报告

- * `target` 和 `targetEntity` 至少提供一个。
- ** 当使用 `targetEntity` 时必填。

JavaScript 示例：

```
const response = await fetch(
  "https://search-alpha.metastar-health.com/prod-api/api/target-assistant/submit",
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
    body: JSON.stringify({
      target: "EGFR",
      language: "zh-CN",
      generateReport: false,
    }),
  },
);
const result = await response.json();
```

Python 示例：

```
url = "https://search-alpha.metastar-health.com/prod-api/api/target-assistant/submit"
headers = {
    "Content-Type": "application/json",
    "x-app-key": "your_app_key",
    "x-app-secret": "your_app_secret",
}
payload = {"target": "EGFR", "language": "zh-CN", "generateReport": False}

response = requests.post(url, headers=headers, json=payload)
result = response.json()
```

成功响应（靶点验证通过）：

```
{
  "data": {
    "success": true,
    "taskId": "550e8400-e29b-41d4-a716-446655440000",
    "message": "Task submitted successfully. Poll /result/:taskId for data readiness, then use /rag/search for context retrieval.",
    "generateReport": false,
    "validatedTarget": {
      "name": "EGFR",
      "aliases": ["ERBB1", "HER1", "ErbB-1"]
    }
  },
  "error": 0
}
```

响应（靶点验证未通过）：

```
{
  "data": {
    "success": false,
    "message": "数据库未收录与\"xyz123\"匹配的靶点数据。最近的结果为：EGFR、BRAF，但均判断为不同靶点。"
  },
  "error": 0
}
```

注意：靶点验证未通过时 HTTP 状态码仍为 200，`error` 仍为 0，需通过 `data.success` 判断是否提交成功。

响应字段说明：

字段	类型	说明
<code>data.success</code>	boolean	任务是否提交成功
<code>data.taskId</code>	string	任务 ID (UUID) ，仅 <code>success=true</code> 时返回
<code>data.message</code>	string	状态描述信息
<code>data.generateReport</code>	boolean	本次任务是否启用平台报告生成
<code>data.validatedTarget</code>	object	验证后的靶点信息，仅 <code>success=true</code> 时返回
<code>data.validatedTarget.name</code>	string	靶点名称
<code>data.validatedTarget.aliases</code>	string[]	靶点别名列表

6.2 查询靶点研究结果

GET /api/target-assistant/result/:taskId

路径参数：

参数名	类型	必填	说明
taskId	string	是	提交任务时返回的任务 ID

响应示例（处理完成）：

默认 generateReport=false 时，任务在数据采集/筛选完成后进入 completed，不会返回报告文件 URL：

```
{
  "data": {
    "taskId": "550e8400-e29b-41d4-a716-446655440000",
    "status": "completed",
    "progress": 100,
    "target": "EGFR",
    "generateReport": false,
    "createdAt": "2026-02-10T18:00:00.000+08:00",
    "updatedAt": "2026-02-10T18:12:30.000+08:00"
  },
  "error": 0
}
```

当提交时传入 generateReport=true，完成态会额外返回平台生成的报告产物 URL：

```
{
  "data": {
    "taskId": "550e8400-e29b-41d4-a716-446655440000",
    "status": "completed",
    "progress": 100,
    "target": "EGFR",
    "generateReport": true,
    "pdfUrl": "https://oss.example.com/reports/EGFR_report.pdf",
    "reportUrl": "https://oss.example.com/reports/EGFR_report.html",
    "bulletJsonUrl": "https://oss.example.com/reports/EGFR_bullet_points.json",
    "referencesCsvUrl": "https://oss.example.com/reports/EGFR_references.csv",
    "createdAt": "2026-02-10T18:00:00.000+08:00",
    "updatedAt": "2026-02-10T18:15:30.000+08:00"
  },
  "error": 0
}
```

结果字段说明：

字段	类型	说明
<code>data.taskId</code>	string	任务 ID
<code>data.status</code>	string	任务状态（见下方状态说明）
<code>data.progress</code>	number	进度百分比（0-100）
<code>data.target</code>	string	靶点名称
<code>data.generateReport</code>	boolean	本次任务是否启用平台报告生成
<code>data.pdfUrl</code>	string	PDF 下载链接（仅 <code>generateReport=true</code> 且报告导出成功时返回）
<code>data.reportUrl</code>	string	HTML 在线报告链接（仅 <code>generateReport=true</code> 且报告导出成功时返回）
<code>data.bulletJsonUrl</code>	string	拆解到章节 / slide / bullet point 的 JSON 文件下载链接（仅 <code>generateReport=true</code> 时可能返回）
<code>data.referencesCsvUrl</code>	string	报告参考文献列表 CSV 文件下载链接（仅 <code>generateReport=true</code> 时可能返回）
<code>data.error</code>	string	错误信息（仅 <code>failed</code> 状态）
<code>data.createdAt</code>	string	任务创建时间（北京时间，ISO 8601， <code>+08:00</code> ）
<code>data.updatedAt</code>	string	最后更新时间（北京时间，ISO 8601， <code>+08:00</code> ）

`bulletJsonUrl` 文件结构说明（仅 `generateReport=true` 时返回）：

该 URL 指向完整报告的结构化 JSON 文件，适合外部系统下载后做二次解析、入库或展示。核心结构如下：

```
{
  "target": "EGFR",
  "language": "en-US",
  "generatedAt": "2026-02-10T18:15:30.000+08:00",
  "chapters": [
    {
      "chapterId": "1",
      "title": "1. Target Introduction",
      "slides": [
        {
          "slideIndex": 1,
          "title": "1.1 Basic Information",
          "bulletPoints": [
            {
              "point_title": "Biological category and significance",
              "point_content": "该 point 对应的具体正文内容"
            }
          ],
          "sources": ["PMID: 12345678"],
          "images": ["https://oss.example.com/reports/image.png"]
        }
      ]
    }
  ]
}
```

- `chapters[]`：报告章节列表，顺序与 HTML/PDF 报告一致

- `slides[]`: 章节下的 slide 列表
- `bulletPoints[]`: 按 point 拆分后的结构化内容, `point_title` 为 point 标题, `point_content` 为对应完整正文
- `sources[]` / `images[]`: 该 slide 关联的引用来源和图片 URL

任务状态流转:

默认 `generateReport=false`:

```
researching --> completed
      \
      \-> failed
```

启用 `generateReport=true`:

```
researching --> generating_report --> generating_pdf --> completed
      \              \              \
      \-> failed      \-> failed      \-> failed
```

状态	说明	进度范围
<code>researching</code>	研究模块数据采集中	0% - 60%
<code>generating_report</code>	报告生成中 (仅 <code>generateReport=true</code>)	60% - 85%
<code>generating_pdf</code>	PDF 转换中 (仅 <code>generateReport=true</code>)	85% - 95%
<code>completed</code>	任务完成	100%
<code>failed</code>	任务失败	-

注意事项:

- 建议轮询间隔: 10 秒
- 数据采集模式通常需要 10 - 30 分钟; 启用平台报告生成时整体可能需要 10 - 50 分钟 (取决于靶点复杂度)
- 每次调用 `submit` 都会创建新任务 (不会去重), 如需避免重复提交请在客户端侧控制
- 默认不生成平台报告。数据采集完成后, 可调用 `POST /api/target-assistant/rag/search` 获取模块数据、报告上下文 (如有) 和全文证据片段, 由调用方使用自己的 pipeline 和 token 生成报告。
- 只有提交时传 `generateReport=true`, 才会返回 `pdfUrl` / `reportUrl` / `bulletJsonUrl` / `referencesCsvUrl` 等平台报告产物。
- 语言支持: `zh-CN` (中文, 默认)、`en-US` (英文)

6.3 靶点报告 QA 问答

```
POST /api/target-assistant/report/qa
```

Content-Type: `application/json`

基于已完成且已生成平台报告的靶点助手任务进行问答。服务端默认使用 `full_task` 范围, 会同时利用已有报告内容和已建立索引的数据进行 RAG 召回, 无需调用方指定章节、版本或检索范围。

任务 ID 兼容: `taskId` 支持两种来源: `POST /api/target-assistant/submit` 返回的开放 API 任务 ID, 或靶点助手页面 URL 中的 `target-research` 任务 ID。

前置条件: 如果传入开放 API 任务 ID, 对应任务状态必须为 `completed`, 且提交任务时需要设置 `generateReport=true`。如任务使用默认 `generateReport=false`, 请改用 `POST /api/target-`

`assistant/rag/search` 获取上下文后在客户端自行生成回答。

请求参数：

字段	类型	必填	默认值	说明
<code>taskId</code>	string	是	-	靶点助手任务 ID；支持 <code>submit</code> 返回的开放 API 任务 ID，或靶点助手页面 URL 中的 <code>target-research</code> 任务 ID
<code>question</code>	string	是	-	用户问题，最长 4000 字符
<code>sessionId</code>	string	否	自动创建	已有 QA 会话 ID；传入后会继续同一轮对话上下文
<code>language</code>	string	否	任务语言	回答语言（ <code>zh-CN</code> 或 <code>en-US</code> ）
<code>history</code>	object[]	否	<code>[]</code>	客户端额外传入的短历史，最多 10 条

history 对象格式：

字段	类型	必填	说明
<code>role</code>	string	是	<code>user</code> 或 <code>assistant</code>
<code>content</code>	string	是	消息内容，单条最长 4000 字符

JavaScript 示例：

```
const response = await fetch(
  "https://search-alpha.metastar-health.com/prod-api/api/target-assistant/report/qa",
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
    body: JSON.stringify({
      taskId: "550e8400-e29b-41d4-a716-446655440000",
      question: "EGFR 作为靶点最强的临床证据是什么？",
      language: "zh-CN",
    }),
  },
);
const result = await response.json();
console.log(result.data.answer);
```

Python 示例：

```
import requests

url = "https://search-alpha.metastar-health.com/prod-api/api/target-assistant/report/qa"
headers = {
  "Content-Type": "application/json",
  "x-app-key": "your_app_key",
```

```

    "x-app-secret": "your_app_secret",
  }
  payload = {
    "taskId": "550e8400-e29b-41d4-a716-446655440000",
    "question": "EGFR 作为靶点最强的临床证据是什么？",
    "language": "zh-CN",
  }

  response = requests.post(url, headers=headers, json=payload)
  result = response.json()
  answer = result["data"]["answer"]

```

成功响应：

```

{
  "data": {
    "success": true,
    "taskId": "550e8400-e29b-41d4-a716-446655440000",
    "target": "EGFR",
    "scope": "full_task",
    "sessionId": "qa_session_1715572800000_abcd1234",
    "answer": "EGFR 的临床证据主要来自多项已上市 EGFR 抑制剂在 EGFR 突变 NSCLC 中的疗效验证...",
    "citations": [
      {
        "id": "1",
        "title": "Osimertinib in untreated EGFR-mutated advanced non-small-cell lung cancer",
        "sourceUrl": "https://pubmed.ncbi.nlm.nih.gov/29151359/",
        "sourceLabel": "PMID:29151359"
      }
    ]
  },
  "error": 0
}

```

响应字段说明：

字段	类型	说明
data.success	boolean	是否回答成功
data.taskId	string	本次请求使用的任务 ID
data.target	string	靶点名称
data.scope	string	固定为 full_task
data.sessionId	string	QA 会话 ID，可用于下一轮连续提问
data.answer	string	Markdown 格式回答
data.citations	array	回答引用来源，包含标题、链接、来源标签等

常见错误：

```
{
  "message": "Target assistant report is not ready. Current status: generating_report",
  "error": 10001
}
```

6.4 靶点助手 RAG 上下文检索

POST /api/target-assistant/rag/search

Content-Type: application/json

基于已完成的靶点助手任务，按查询语义召回两类上下文：

- `relatedData`：靶点助手各模块的相关结构化数据（生物学、疾病、竞争格局、临床前、临床、洞见等）
- `reportChunks`：已生成报告正文中的相关 chunks
- `relatedData[].attachedFulltextChunks`：当召回项可匹配到平台全文库时，会附带全文片段，便于调用方做证据追溯或二次 RAG 编排。默认返回与问题相关的排序/筛选后片段；如 `includeAllFulltextChunks=true`，则返回对应 PMID 的全部可用全文 chunks，由调用方自行排序。

该接口**不生成回答**，只返回 RAG 召回结果，适合外部系统自行做问答、摘要、引用追溯或二次分析。如需服务端直接生成回答，请使用 `POST /api/target-assistant/report/qa`。

全文 chunk 说明：当前全文挂载适用于 `biology` / `disease` / `preclinical` 等文献型模块。默认模式会由服务端进行 chunk 召回、排序、去重和预算裁剪；`includeAllFulltextChunks=true` 时跳过服务端 chunk rerank/筛选，直接返回对应文献的全部可用全文 chunks。

任务 ID 兼容：`taskId` 支持两种来源：`POST /api/target-assistant/submit` 返回的开放 API 任务 ID，或靶点助手页面 URL 中的 `target-research` 任务 ID。

前置条件：如果传入开放 API 任务 ID，对应任务状态必须为 `completed`。默认 `generateReport=false` 的任务也可以调用本接口，此时 `reportChunks` 通常为空，`relatedData` 和全文证据片段仍可用于外部 RAG 编排。

请求参数：

字段	类型	必填	默认值	说明
<code>taskId</code>	string	是	-	靶点助手任务 ID；支持开放 API 任务 ID 或 <code>target-research</code> 任务 ID
<code>question</code>	string	是	-	检索问题或关键词，最长 4000 字符
<code>moduleId</code>	string	否	-	限定模块数据检索范围： <code>biology</code> / <code>disease</code> / <code>competition</code> / <code>preclinical</code> / <code>clinical</code> / <code>insights</code> / <code>supplementary</code>
<code>dataLimit</code>	number	否	10	返回相关模块数据条数，范围 1-50
<code>reportLimit</code>	number	否	10	返回报告 chunks 条数，范围 1-50
<code>includeAllFulltextChunks</code>	boolean	否	false	是否返回每条文献型 <code>relatedData</code> 对应 PMID 的全部可用全文 chunks。默认 <code>false</code> 时返回服务端排序/筛选后的相关 chunks；设为 <code>true</code> 时不做服务端 chunk rerank/筛选，调用方可自行排序

JavaScript 示例：

```

const response = await fetch(
  "https://search-alpha.metastar-health.com/prod-api/api/target-assistant/rag/search",
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
    body: JSON.stringify({
      taskId: "550e8400-e29b-41d4-a716-446655440000",
      question: "EGFR 的临床疗效和耐药风险证据",
      dataLimit: 8,
      reportLimit: 6,
      includeAllFulltextChunks: false,
    }),
  },
);
const result = await response.json();
console.log(result.data.relatedData);
console.log(result.data.reportChunks);

```

Python 示例:

```

import requests

url = "https://search-alpha.metastar-health.com/prod-api/api/target-assistant/rag/search"
headers = {
    "Content-Type": "application/json",
    "x-app-key": "your_app_key",
    "x-app-secret": "your_app_secret",
}
payload = {
    "taskId": "550e8400-e29b-41d4-a716-446655440000",
    "question": "EGFR 的临床疗效和耐药风险证据",
    "dataLimit": 8,
    "reportLimit": 6,
    "includeAllFulltextChunks": false,
}

response = requests.post(url, headers=headers, json=payload)
result = response.json()
related_data = result["data"]["relatedData"]
report_chunks = result["data"]["reportChunks"]

```

成功响应:

```

{
  "data": {
    "success": true,
    "taskId": "550e8400-e29b-41d4-a716-446655440000",
    "target": "EGFR",
    "question": "EGFR 的临床疗效和耐药风险证据",
    "relatedData": [
      {
        "id": "clinical:trial-001",
        "type": "data",

```

```
"moduleId": "clinical",
"title": "Osimertinib in EGFR-mutated NSCLC",
"content": "召回到的模块数据内容摘要或结构化文本...",
"sourceUrl": "https://pubmed.ncbi.nlm.nih.gov/29151359/",
"sourceLabel": "PMID:29151359",
"evidenceGrade": "high",
"dataItemId": "trial-001",
"isAIRecommended": true,
"isSelected": false,
"fulltextAttachment": {
  "status": "matched",
  "pmid": "29151359",
  "sourceIndexes": [ "platform_fulltext" ],
  "chunkCount": 42,
  "attachedChunkCount": 2,
  "hasFigures": true,
  "hasTables": false
},
"attachedFulltextChunks": [
  {
    "chunkId": "fulltext:29151359:8",
    "sourceIndex": "platform_fulltext",
    "parentPmid": "29151359",
    "sectionKey": "Results",
    "sectionType": "results",
    "chunkType": "fulltext",
    "order": 8,
    "content": "与本次问题相关的全文结果段落或证据片段..."
  },
  {
    "chunkId": "fulltext:29151359:figure-1",
    "sourceIndex": "platform_fulltext",
    "parentPmid": "29151359",
    "sectionKey": "Figure 1",
    "sectionType": "figure",
    "chunkType": "figure",
    "order": 12,
    "content": "{ \"type\": \"image\", \"image_caption\": \"与本次问题相关的图注证据...\" }"
  }
]
},
"reportChunks": [
  {
    "id": "report:v1:3",
    "type": "report_chunk",
    "moduleId": "report",
    "title": "Clinical validation and resistance risks",
    "content": "召回到的报告正文 chunk...",
    "sourceLabel": "report",
    "reportSectionId": "clinical",
    "reportSlideId": "clinical-2"
  }
],
"total": {
  "relatedData": 1,
  "reportChunks": 1
}
```

```
    },
    "error": 0
}
```

响应字段说明:

字段	类型	说明
data.success	boolean	是否检索成功
data.taskId	string	本次请求使用的任务 ID
data.target	string	靶点名称
data.question	string	本次检索问题
data.relatedData	array	相关模块数据列表
data.reportChunks	array	相关报告正文 chunks
data.total.relatedData	number	返回的模块数据条数
data.total.reportChunks	number	返回的报告 chunk 条数

relatedData[] / reportChunks[] 对象字段:

字段	类型	说明
id	string	召回项 ID
type	string	data 或 report_chunk
moduleId	string	所属模块; 报告 chunk 固定为 report
title	string	标题
content	string	召回内容; 超长内容会被服务端截断
sourceUrl	string	来源链接 (如有)
sourceLabel	string	来源标签, 如 PMID、trial ID、report
evidenceGrade	string	证据等级 (模块数据可能返回)
dataItemId	string	模块数据原始条目 ID (仅 relatedData)
reportSectionId	string	报告章节 ID (仅 reportChunks)
reportSlideId	string	报告 slide ID (仅 reportChunks)
isAIRecommended	boolean	是否 AI 推荐数据 (仅 relatedData)
isSelected	boolean	是否用户选中数据 (仅 relatedData)
fulltextAttachment	object	全文片段挂载状态 (仅可能出现在 relatedData)
attachedFulltextChunks	array	与召回项关联的全文证据片段; 未匹配到全文时为空数组

fulltextAttachment 对象字段:

字段	类型	说明
status	string	全文挂载状态： <code>matched</code> / <code>unmatched</code> / <code>no_identifier</code> / <code>not_applicable</code> / <code>skipped</code> / <code>error</code>
pmid	string	关联 PMID（如有）
sourceIndexes	string[]	命中的全文来源标识
chunkCount	number	候选全文片段数量
attachedChunkCount	number	最终返回的全文片段数量
hasFigures	boolean	候选全文中是否存在图/图片片段
hasTables	boolean	候选全文中是否存在表格片段
error	string	全文挂载失败原因（仅 <code>status = error</code> 时可能出现）

`attachedFulltextChunks[]` 对象字段：

字段	类型	说明
chunkId	string	全文片段 ID
sourceIndex	string	片段来源标识
parentPmid	string	所属文献 PMID
sectionKey	string	原始段落标题或图表标题
sectionType	string	归一化段落类型： <code>results</code> / <code>figure</code> / <code>table</code> / <code>evidence_body</code> / <code>abstract</code> / <code>methods</code> / <code>discussion</code> / <code>intro</code> / <code>other</code>
chunkType	string	原始 chunk 类型： <code>fulltext</code> / <code>figure</code> / <code>table</code> / <code>other</code>
order	number	片段在全文中的顺序
content	string	全文片段正文；图表片段可能为 JSON 字符串，包含图片地址和图注

常见错误：

```
{
  "message": "Target assistant report is not ready. Current status: generating_report",
  "error": 10001
}
```

7. 靶点快速评估

批量靶点快速评估服务，支持同时提交多个靶点（最多 10 个）。

- 接口类型： 异步任务（提交 + 轮询）
- Content-Type: `application/json`
- 典型耗时： 1 - 5 分钟/靶点（批量并行，整体取决于最慢的靶点）
- 超时上限： 30 分钟
- 建议轮询间隔： 5 - 10 秒
- 并发限制： 5 个/秒（每次最多 10 个靶点，自动去重）

结果保留： 7 天
依赖外部 LLM： 是（Claude Sonnet 4 生成评估报告）

7.1 提交靶点快速评估任务

POST /api/target-quick-assessment/submit

Content-Type: application/json

请求参数：

字段	类型	必填	说明
targetNames	string[]	是	靶点基因 ID 列表，1-10 个，如 ["EGFR", "CDK4"]

JavaScript 示例：

```
const response = await fetch(
  "https://search-alpha.metastar-health.com/prod-api/api/target-quick-assessment/submit",
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
    body: JSON.stringify({
      targetNames: ["EGFR", "CDK4"],
    }),
  },
);
const result = await response.json();
// result.data.taskId
```

Python 示例：

```
url = "https://search-alpha.metastar-health.com/prod-api/api/target-quick-assessment/submit"
headers = {
  "Content-Type": "application/json",
  "x-app-key": "your_app_key",
  "x-app-secret": "your_app_secret",
}
payload = {"targetNames": ["EGFR", "CDK4"]}

response = requests.post(url, headers=headers, json=payload)
result = response.json()
task_id = result["data"]["taskId"]
```

成功响应：

```
{
  "data": {
    "taskId": "2e207c97-cbb9-47e3-9251-42cf7110a8e4"
  },
  "error": 0
}
```

错误响应（参数无效）：

```
{
  "message": "无效的请求参数：targetNames",
  "error": 1001
}
```

7.2 查询靶点快速评估结果

GET /api/target-quick-assessment/result/:taskId

路径参数：

参数名	类型	必填	说明
taskId	string	是	提交任务时返回的任务 ID

响应示例（处理中）：

```
{
  "data": {
    "taskId": "2e207c97-cbb9-47e3-9251-42cf7110a8e4",
    "status": "processing",
    "progress": 50,
    "completedCount": 1,
    "totalCount": 2,
    "targets": [
      {
        "target": "EGFR",
        "status": "completed",
        "progress": 100,
        "result": { "reportContent": "..."}
      },
      {
        "target": "CDK4",
        "status": "processing",
        "progress": 45
      }
    ],
    "createdAt": "2026-04-08T04:45:00.000Z",
    "updatedAt": "2026-04-08T04:46:00.000Z"
  },
  "error": 0
}
```

响应示例（全部完成）：

```
{
  "data": {
    "taskId": "2e207c97-cbb9-47e3-9251-42cf7110a8e4",
    "status": "completed",
    "progress": 100,
    "completedCount": 2,
    "totalCount": 2,
    "targets": [
      {
        "target": "EGFR",
        "status": "completed",
        "progress": 100,
        "result": {
          "reportContent": "# EGFR 靶点快速评估报告\n\n..."
        }
      },
      {
        "target": "CDK4",
        "status": "completed",
        "progress": 100,
        "result": {
          "reportContent": "# CDK4 靶点快速评估报告\n\n..."
        }
      }
    ],
    "createdAt": "2026-04-08T04:45:00.000Z",
    "updatedAt": "2026-04-08T04:46:30.000Z"
  },
  "error": 0
}
```

批量结果字段说明：

字段	类型	说明
taskId	string	批量任务 ID
status	string	批量总状态（processing / completed / failed）
progress	number	总进度（0-100），按完成靶点数计算
completedCount	number	已完成（含失败）的靶点数
totalCount	number	总靶点数
targets	array	各靶点的独立结果

单个靶点结果字段说明：

字段	类型	说明
<code>target</code>	string	靶点基因 ID
<code>status</code>	string	该靶点状态（ <code>pending</code> / <code>processing</code> / <code>completed</code> / <code>failed</code> ）
<code>progress</code>	number	该靶点进度（0-100）
<code>result</code>	object	评估结果（仅 <code>completed</code> 状态）
<code>result.reportContent</code>	string	Markdown 格式的完整评估报告
<code>error</code>	string	错误信息（仅 <code>failed</code> 状态）

注意事项：

- 每次最多提交 10 个靶点
- 重复的靶点名称会自动去重（大小写不敏感）
- 单个靶点处理时间约 1-5 分钟，取决于管线数据量和 LLM 响应速度
- 建议轮询间隔：5-10 秒
- `reportContent` 为 Markdown 格式，可直接渲染为 HTML

8. 药物信息查询

提供面向外部集成场景的药物检索能力，支持按疾病、药物、靶点、公司、开发阶段、药物模态等多维条件组合搜索。接口仅接受结构化对象参数，适合直接承接自动补全接口返回的实体对象。返回结果已按前端表格展示格式进行标准化，并附带英文表头，便于直接渲染列表。

接口类型：同步（实时返回）

Content-Type: `application/json`

典型耗时：1 - 3 秒

响应超时：30 秒

并发限制：10 个/秒

数据源：Elasticsearch 药物索引（实时）

8.1 搜索药物

```
POST /api/drugs/search
Content-Type: application/json
```

请求体参数：

字段	类型	必填	默认值	说明
diseases	object[]	否*	[]	疾病 / 适应症实体列表
drugs	object[]	否*	[]	药物实体列表
targets	object[]	否*	[]	靶点实体列表
companies	object[]	否*	[]	公司实体列表
statuses	string[]	否*	[]	开发阶段列表
modalities	string[]	否*	[]	药物模态列表
limit	integer	否	20	返回结果数量 (1-1000)

* 至少需要提供一个搜索条件 (diseases、drugs、targets、companies、statuses、modalities 中的任意一个)。

实体对象格式：

字段	类型	必填	说明
name	string	是	实体主名称，可直接使用自动补全接口返回的 name
aliases	string[] string	否	实体别名；支持数组，也支持自动补全接口返回的分号分隔字符串

搜索逻辑：

- targets 中多个对象之间为“与”关系
- 单个 target 对象内部的 name + aliases 为“或”关系
- diseases、drugs、companies 中多个对象及其 aliases 会展开为同类字段内的“或”关系
- 不同维度之间为“与”关系
- 返回结果字段与前端药物表格展示列保持一致，并额外返回 columns 作为英文表头定义
- 调用方可直接复用自动补全返回对象，额外字段会被忽略

JavaScript 示例：

```
const body = {
  targets: [
    {
      name: "GLP1R",
      aliases: "GLP-1R;Glucagon-like peptide-1 receptor",
    },
  ],
  diseases: [
    { name: "Type 2 Diabetes", aliases: ["T2D"] },
    { name: "Obesity" },
  ],
  statuses: ["Launched"],
  limit: 20,
};

const response = await fetch(
  "https://search-alpha.metastar-health.com/prod-api/api/drugs/search",
  {
```

```

    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
    body: JSON.stringify(body),
  },
);

const result = await response.json();

```

Python 示例:

```

import requests

url = "https://search-alpha.metastar-health.com/prod-api/api/drugs/search"
headers = {
  "Content-Type": "application/json",
  "x-app-key": "your_app_key",
  "x-app-secret": "your_app_secret",
}
payload = {
  "targets": [
    {
      "name": "GLP1R",
      "aliases": "GLP-1R;Glucagon-like peptide-1 receptor"
    }
  ],
  "diseases": [
    {"name": "Type 2 Diabetes", "aliases": ["T2D"]},
    {"name": "Obesity"}
  ],
  "statuses": ["Launched"],
  "limit": 20,
}

response = requests.post(url, headers=headers, json=payload)
result = response.json()

```

成功响应:

```

{
  "error": 0,
  "data": {
    "columns": [
      { "key": "genericDrugName", "title": "Generic Drug Name" },
      { "key": "chemicalStructure", "title": "Chemical Structure" },
      { "key": "mechanismOfAction", "title": "Mechanism Of Action" },
      { "key": "company", "title": "Company" },
      { "key": "disease", "title": "Disease" },
      { "key": "highestIndicationStage", "title": "Highest Indication Stage" },
      { "key": "highestPipelineStage", "title": "Highest Pipeline Stage" },
      { "key": "drugCountry", "title": "Drug Country" },
      { "key": "modality", "title": "Modality" },
      { "key": "deliveryRoute", "title": "Delivery Route" },
      { "key": "target", "title": "Target" }
    ]
  }
}

```

```
],
"rows": [
  {
    "genericDrugName": "Semaglutide",
    "chemicalStructure": "CC(C)CC1=CC=CC=C1",
    "mechanismOfAction": "GLP-1 receptor agonist",
    "company": "Novo Nordisk",
    "disease": ["Type 2 Diabetes", "Obesity"],
    "highestIndicationStage": "Launched",
    "highestPipelineStage": "Launched",
    "drugCountry": ["US", "EU"],
    "modality": "Peptide",
    "deliveryRoute": "Subcutaneous",
    "target": "GLP1R"
  }
],
"total": 126,
"limit": 20
},
"query": {
  "diseases": [
    { "name": "Type 2 Diabetes", "aliases": ["T2D"] },
    { "name": "Obesity", "aliases": [] }
  ],
  "drugs": [],
  "targets": [
    {
      "name": "GLP1R",
      "aliases": ["GLP-1R", "Glucagon-like peptide-1 receptor"]
    }
  ],
  "companies": [],
  "statuses": ["Launched"],
  "modalities": [],
  "limit": 20
},
"message": "ok"
}
```

顶层字段说明：

字段	类型	说明
<code>error</code>	number	错误码， <code>0</code> 表示成功
<code>data</code>	object	表格化药物结果，包含表头定义与数据行
<code>query</code>	object	服务端回显的规范化对象参数
<code>message</code>	string	人类可读消息，成功时通常为 <code>ok</code>

`data.columns` 字段说明：

字段	类型	说明
key	string	稳定字段键名，适合程序渲染
title	string	英文表头，和前端 i18n 英文展示一致

data.rows 字段说明：

字段	类型	说明
genericDrugName	string string[] null	药物名称
chemicalStructure	string string[] null	化学结构 / SMILES
mechanismOfAction	string string[] null	作用机制
company	string string[] null	公司名称（优先返回标准化名称）
disease	string string[] null	疾病 / 适应症（优先返回标准化名称）
highestIndicationStage	string string[] null	适应症最高开发阶段
highestPipelineStage	string string[] null	管线最高研发阶段
drugCountry	string string[] null	药物开发国家
modality	string string[] null	药物模态
deliveryRoute	string string[] null	给药途径
target	string string[] null	靶点信息

query 字段说明：

字段	类型	说明
diseases	object[]	规范化后的疾病实体列表
drugs	object[]	规范化后的药物实体列表
targets	object[]	规范化后的靶点实体列表
companies	object[]	规范化后的公司实体列表
statuses	string[]	规范化后的开发阶段条件
modalities	string[]	规范化后的模态条件
limit	integer	实际生效的返回条数上限

常见查询示例：

```
{
  "drugs": [{ "name": "Semaglutide", "aliases": ["Ozempic", "Rybelsus"] }],
  "limit": 10
}
```

错误响应（缺少搜索条件）：

```
{
  "statusCode": 400,
  "message": "At least one search condition is required: diseases, drugs, targets, companies, statuses, or modalities",
  "error": "Bad Request"
}
```

注意事项:

- 单次查询最多返回 1000 条结果
- 为了保证表格一致性, 返回字段已经按前端展示列做了标准化, 不再直接暴露全部原始 ES 字段
- 某些列可能返回 `string`、`string[]` 或 `null`, 具体取决于底层数据是否存在多值
- 推荐先调用 `GET /api/ai/autocomplete/entities` 获取实体, 再将返回对象中的 `name` / `aliases` 透传给本接口

9. 文献临床前数据检索

提供面向外部集成场景的文献来源临床前 tool 数据检索能力, 支持按靶点、疾病、公司组合搜索。接口仅接受结构化对象参数, 适合直接承接自动补全接口返回的实体对象。返回结果按文档对象分页, 不展开 `model_info` / `study`; 每个对象返回文档级字段, 并保留原始 `model_info` 供详情展示使用。

接口类型: 同步 (实时返回)

Content-Type: `application/json`

典型耗时: 1 - 3 秒

响应超时: 30 秒

并发限制: 10 个/秒

数据源: Elasticsearch 文献临床前索引 (实时)

9.1 搜索文献临床前数据

```
POST /api/preclinical-literature/search
Content-Type: application/json
```

请求体参数:

字段	类型	必填	默认值	说明
<code>targets</code>	<code>object[]</code>	否*	<code>[]</code>	靶点实体列表
<code>diseases</code>	<code>object[]</code>	否*	<code>[]</code>	疾病 / 适应症实体列表
<code>companies</code>	<code>object[]</code>	否*	<code>[]</code>	公司实体列表
<code>page</code>	<code>integer</code>	否	1	页码, 从 1 开始
<code>pageSize</code>	<code>integer</code>	否	50	每页返回文档对象数量 (1-100)

* 至少需要提供一个搜索条件 (`targets`、`diseases`、`companies` 中的任意一个)。

实体对象格式:

字段	类型	必填	说明
name	string	是	实体主名称，可直接使用自动补全接口返回的 name
aliases	string[] string	否	实体别名；支持数组，也支持自动补全接口返回的分号分隔字符串

搜索逻辑：

- targets 中多个对象之间为“与”关系
- 单个 target 对象内部的 name + aliases 为“或”关系
- diseases、companies 中多个对象及其 aliases 会展开为同类字段内的“或”关系
- 不同维度之间为“与”关系
- 调用方可直接复用自动补全返回对象，额外字段会被忽略

JavaScript 示例：

```
const body = {
  targets: [
    {
      name: "GLP1R",
      aliases: "GLP-1R;Glucagon-like peptide-1 receptor",
    },
  ],
  diseases: [{ name: "Type 2 Diabetes", aliases: ["T2D"] }],
  companies: [{ name: "Novo Nordisk" }],
  page: 1,
  pageSize: 50,
};

const response = await fetch(
  "https://search-alpha.metastar-health.com/prod-api/api/preclinical-literature/search",
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
    body: JSON.stringify(body),
  },
);

const result = await response.json();
```

Python 示例：

```
import requests

url = "https://search-alpha.metastar-health.com/prod-api/api/preclinical-literature/search"
headers = {
    "Content-Type": "application/json",
    "x-app-key": "your_app_key",
    "x-app-secret": "your_app_secret",
}
payload = {
```

```

    "targets": [
      {
        "name": "GLP1R",
        "aliases": "GLP-1R;Glucagon-like peptide-1 receptor"
      }
    ],
    "diseases": [
      { "name": "Type 2 Diabetes", "aliases": ["T2D"]}
    ],
    "companies": [
      { "name": "Novo Nordisk" }
    ],
    "page": 1,
    "pageSize": 50,
  }

response = requests.post(url, headers=headers, json=payload)
result = response.json()

```

成功响应:

```

{
  "error": 0,
  "data": {
    "status": "completed",
    "progress": 100,
    "result": [
      {
        "PMID": "12345678",
        "title": "Preclinical evaluation of GLP1R agonism in metabolic disease models",
        "Target": "GLP1R",
        "Antibody_Name": "Semaglutide",
        "Clinical_Phase": "Approved",
        "source_type": "literature",
        "pubdate": "2024-01-02",
        "indication": "Type 2 Diabetes",
        "model_info": [
          {
            "model_name": "db/db mouse",
            "Species": "Mouse",
            "extracted_fields": {
              "base_characteristics": {
                "Experiment_Type": "In vivo",
                "Species": "Mouse"
              },
              "experimental_studies": []
            }
          }
        ]
      }
    ],
    "count": 1,
    "page": 1,
    "pageSize": 50,
    "hasMore": true,
    "nextPage": 2
  }
}

```

Poster 来源示例 (CDH17) :

以下请求按 CDH17 靶点检索，返回结果中可能包含 `source_type: "poster"` 的 poster 来源记录。

```
{
  "targets": [
    {
      "name": "CDH17",
      "aliases": "cadherin 17;CAD17_HUMAN;CDH17;CDH17 variant 1;CDH17 variant 2;Cadherin 17, LI  
Cadherin (Liver-Intestine);Cadherin 17, transcript variant 1;Cadherin 17, transcript variant  
2;Cadherin-17;Cadherin-17 (isoform 1);Ensembl:  
ENSG00000079112;GC08M090347;GC08M093885;GC08M094808;GC08M094985;GC08M095095;GC08M095208;HPT-1;HPT-1  
Cadherin;HPT1;Human Intestinal Peptide-Associated Transporter HPT-1;Human Peptide Transporter;Human  
peptide transporter 1;Intestinal Peptide-Associated Transporter HPT-1;LI Cadherin;LI-Cadherin;Liver-  
Intestine Cadherin;UniProtKB/Swiss-Prot: Q12864;cadherin 17 (CDH17)"
    }
  ],
  "page": 1,
  "pageSize": 5
}
```

Poster 来源成功响应节选:

为便于阅读，下方仅保留一个 `source_type: "poster"` 的 `result` 示例和一个 `model_info` 示例对象；实际响应会按 `page / pageSize` 返回完整文档对象。

```
{
  "error": 0,
  "data": {
    "status": "completed",
    "progress": 100,
    "result": [
      {
        "Target": "CDH17,CDH17",
        "Antibody_Name": "TAVO307A,TAVO307B",
        "source_type": "poster",
        "title": "Novel ADC and γδ T Cell Engager Targeting CDH17 for the Therapy of Gastrointestinal  
Cancers",
        "pubdate": "2025",
        "indication": "Gastrointestinal cancers; Stomach cancer; Esophageal cancer; Liver cancer;  
Colon cancer; Pancreatic cancer; Neuroendocrine cancer,Gastrointestinal cancers",
        "model_info": [
          {
            "model_purpose": "efficacy_validation",
            "model_name": "Gastrointestinal cancer, cell-line derived xenograft, in mouse",
            "model_condition": [
              "gastric cancer",
              "pancreatic cancer",
              "colon cancer"
            ],
            "model_type": "animal_model",
            "Experiment_Type": "in_vivo",
            "Species": "Mus musculus (mouse)",
            "Strain": "not_found",
            "model_id": "animal_model_1"
          }
        ]
      }
    ]
  }
}
```

```
    }
  ],
  "count": 5,
  "page": 1,
  "pageSize": 5,
  "hasMore": true,
  "nextPage": 2
}
```

顶层字段说明：

字段	类型	说明
error	number	错误码，0 表示成功
data	object	检索结果业务数据；由开放 API 统一外层封装

data 字段说明：

字段	类型	说明
status	string	同步检索状态，成功时为 completed
progress	number	同步检索进度，成功时为 100
result	object[]	文献来源临床前文档对象数组；按文档分页，不展开 model_info / study
count	number	当前页实际返回的文档对象数量
page	integer	当前页码
pageSize	integer	实际生效的每页条数，最大 100
hasMore	boolean	是否可能存在下一页
nextPage	integer	存在下一页时返回下一页页码

result[] 字段说明：

字段	类型	说明
Target	string	靶点
Antibody_Name	string	药物 / 抗体名称
Clinical_Phase	string	当前状态
PMID	string	PubMed ID
source_type	string	来源类型，当前包含 literature、poster、presentation 三种
title	string	文献 / 海报 / 演示文稿标题
pubdate	string	发表日期；由底层 new_date 映射而来，若无 new_date 则使用原始 pubdate
indication	string	疾病 / 适应症
model_info	object[]	原始模型信息数组，用于详情展示；实验类型、模型名称、物种 / 细胞类型、研究名称、测量指标等信息均在该字段内

错误响应（缺少搜索条件）：

```
{
  "statusCode": 400,
  "message": "At least one search condition is required: targets, diseases, or companies",
  "error": "Bad Request"
}
```

注意事项：

- 单页最多返回 100 条文档对象；默认 pageSize=50，需要更多数据时使用 page + pageSize 翻页
- 推荐先调用 GET /api/ai/autocomplete/entities 获取实体，再将返回对象中的 name / aliases 透传给本接口
- source_type 为来源标签，目前可能为 literature、poster、presentation
- result 不做 model_info / study 展开；实验类型、模型名称、物种 / 细胞类型、研究名称、测量指标等信息请从 model_info 中读取
- result[] 会保留原始 model_info 数组，便于调用方展示模型详情；暂不返回 image_info / full_text
- 本接口为同步查询接口，不返回 taskId；如需异步 PDF 文献解析，请使用接口 3

10. HPA 靶点画像查询

查询 Human Protein Atlas (HPA) 中的靶点蛋白表达、亚细胞定位、疾病表达图谱等结构化信息。

接口类型：同步（实时返回）

Content-Type: application/x-www-form-urlencoded（query string）

典型耗时：200 - 800 ms

响应超时：10 秒

并发限制：20 个/秒

数据源：HPA 数据库本地镜像 + OSS 图片资源（图片 CDN 加速）

10.1 查询 HPA 靶点画像

```
GET /api/hpa/profile
```

请求参数：

参数名	类型	必填	说明
target	string	是	靶点基因符号或主名称，例如 BAG3
aliases	string	否	靶点别名，使用英文分号 ; 或逗号 , 分隔，例如 BIS;BAG3_HUMAN

JavaScript 示例：

```
const params = new URLSearchParams({
  target: "BAG3",
  aliases: "BIS;BAG3_HUMAN",
});

const response = await fetch(
  `https://search-alpha.metastar-health.com/prod-api/api/hpa/profile?${params.toString()}`,
  {
    headers: {
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
  },
);

const result = await response.json();
```

Python 示例：

```
import requests

url = "https://search-alpha.metastar-health.com/prod-api/api/hpa/profile"
headers = {
  "x-app-key": "your_app_key",
  "x-app-secret": "your_app_secret",
}
params = {
  "target": "BAG3",
  "aliases": "BIS;BAG3_HUMAN",
}

response = requests.get(url, headers=headers, params=params)
result = response.json()
```

成功响应（命中画像）：

```
{
  "error": 0,
  "data": {
    "target": "BAG3",
    "subcellularHumanCellLines": "https://example.com/hpa/subcellular_human_cell_lines.png",
    "rnaProteinExpressionSummary": "BAG3 shows enriched expression in muscle tissues and stress-responsive cell populations.",
    "tissueProteinExpressionOverview":
      "https://example.com/hpa/tissue_protein_expression_overview.png",
    "tissueRnaExpressionOverview": "https://example.com/hpa/tissue_rna_expression_overview.png",
```

```
    "tissueExpressionCluster": "Muscle / cardiomyocyte enriched",
    "cancerProteinExpressionPanel": "https://example.com/hpa/cancer_protein_expression_panel.png",
    "bloodPanDiseaseSomascan": "https://example.com/hpa/blood_pan_disease_somascan.png",
    "proteinFunction": "Co-chaperone involved in proteostasis, apoptosis regulation, and stress response.",
    "proteinExpressionAndLocalization": "Localized mainly in cytoplasm with context-dependent nuclear enrichment.",
    "cellTypeRnaExpression": "Cardiomyocytes, skeletal myocytes",
    "msId": "ENSG00000151929",
    "msName": "BAG3",
    "targetNames": ["BAG3", "BIS", "BAG3_HUMAN"],
    "recommendedDiseasePanel": {
      "oncologyRelated": false,
      "panelType": "blood",
      "url": "https://example.com/hpa/blood_pan_disease_somascan.png"
    }
  },
  "query": {
    "target": "BAG3",
    "aliases": ["BIS", "BAG3_HUMAN"]
  },
  "message": "ok"
}
```

成功响应（未命中画像）：

```
{
  "error": 0,
  "data": null,
  "query": {
    "target": "UNKNOWN",
    "aliases": []
  },
  "message": "No HPA profile found for target \"UNKNOWN\""
}
```

错误响应（缺少必填参数）：

```
{
  "statusCode": 400,
  "message": "target is required",
  "error": "Bad Request"
}
```

顶层字段说明：

字段	类型	说明
error	number	错误码，0 表示成功
data	object null	HPA 画像数据；未命中时返回 null
query	object	服务端回显的规范化查询参数（当前文档仅说明 target 与 aliases）
message	string	人类可读消息，成功时通常为 ok

data 字段说明：

字段	类型	说明
target	string	HPA 中匹配到的标准靶点名称
subcellularHumanCellLines	string	亚细胞定位图 URL
rnaProteinExpressionSummary	string	RNA/蛋白表达摘要
tissueProteinExpressionOverview	string	正常组织蛋白表达总览图 URL
tissueRnaExpressionOverview	string	正常组织 RNA 表达总览图 URL
tissueExpressionCluster	string	组织表达聚类描述
cancerProteinExpressionPanel	string	癌症蛋白表达图谱 URL
bloodPanDiseaseSomascan	string	泛疾病外周血 SomaScan 表达图谱 URL
proteinFunction	string	蛋白功能摘要
proteinExpressionAndLocalization	string	蛋白表达与定位说明
cellTypeRnaExpression	string	细胞类型 RNA 表达摘要
msId	string	HPA / Ensembl 关联 ID
msName	string	HPA 记录中的名称
targetNames	string[]	命中的别名/同义名集合
recommendedDiseasePanel	object	基于疾病上下文推荐的疾病表达面板

recommendedDiseasePanel 字段说明：

字段	类型	说明
oncologyRelated	boolean	当前疾病上下文是否被识别为肿瘤相关
panelType	string	推荐面板类型，可能为 cancer 或 blood
url	string	推荐面板图片 URL

注意事项：

- 本接口为同步查询接口，适合在前端或外部系统中直接拉取靶点 HPA 画像。
- aliases 支持使用英文分号 ; 或逗号 , 分隔多个值。
- recommendedDiseasePanel 为服务端返回的推荐疾病表达图谱，不影响其他 HPA 字段的返回。
- 若 bloodPanDiseaseSomascan 存在，通常优先推荐 blood 面板；否则回退到 cancerProteinExpressionPanel。

11. GBD 疾病负担数据查询

查询 Global Burden of Disease (GBD) 结构化疾病负担数据，支持按疾病/原因、指标、地区、性别、年龄组、计量方式和年份过滤。接口为同步查询接口，单次最多返回 3000 条数据。

接口类型： 同步（实时返回）
Content-Type: application/json
典型耗时： 1-3 秒（受 limit 影响，3000 条接近 5 秒）
响应超时： 30 秒
并发限制： 10 个/秒
数据源： GBD 结构化数据库本地镜像

11.1 查询 GBD 数据

```
POST /api/gbd/search
Content-Type: application/json
```

请求体参数：

字段	类型	必填	默认值	说明
causeName	string	否*	-	GBD cause / 疾病名称，例如 Diabetes mellitus
measureName	string	否*	-	指标名称，例如 Prevalence、Incidence、Deaths
locationName	string	否*	-	地区名称，例如 Global、China
sexName	string	否*	-	性别，仅支持 Male、Female、Both
metricName	string	否*	-	计量方式，例如 Number、Rate、Percent
year	integer	否*	-	精确年份；传入后优先于 startYear / endYear
startYear	integer	否*	-	起始年份
endYear	integer	否*	-	结束年份
limit	integer	否	100	返回结果数量（1-3000）

* 至少需要提供一个搜索条件（除 limit 外任意一个字段）。

搜索逻辑：

- 多个过滤维度之间为“与”关系。
- causeName、measureName、locationName、metricName 使用 GBD 名称字段匹配。
- sexName 对外只接受英文枚举 Male、Female、Both；服务端会自动映射到底层 GBD 中英文值（Male/Female/Both 与 男/女/合计），返回时统一归一化为英文。
- year 用于精确年份匹配；如未传 year，则使用 startYear / endYear 做年份范围过滤。
- 返回结果按 year 倒序排列。
- 单次请求最大返回 3000 条，即使底层命中总数更高也只返回前 3000 条。

JavaScript 示例：

```
const body = {
  causeName: "Diabetes mellitus",
  measureName: "Prevalence",
  locationName: "Global",
  sexName: "Both",
  metricName: "Rate",
  startYear: 2019,
  endYear: 2021,
```

```

    limit: 100,
  };

const response = await fetch(
  "https://search-alpha.metastar-health.com/prod-api/api/gbd/search",
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
    body: JSON.stringify(body),
  },
);

const result = await response.json();

```

Python 示例:

```

import requests

url = "https://search-alpha.metastar-health.com/prod-api/api/gbd/search"
headers = {
    "Content-Type": "application/json",
    "x-app-key": "your_app_key",
    "x-app-secret": "your_app_secret",
}
payload = {
    "causeName": "Diabetes mellitus",
    "measureName": "Prevalence",
    "locationName": "Global",
    "sexName": "Both",
    "metricName": "Rate",
    "startYear": 2019,
    "endYear": 2021,
    "limit": 100,
}

response = requests.post(url, headers=headers, json=payload)
result = response.json()

```

成功响应:

```

{
  "error": 0,
  "data": {
    "rows": [
      {
        "causeId": 587,
        "causeName": "Diabetes mellitus",
        "measureId": 5,
        "measureName": "Prevalence",
        "locationId": 1,
        "locationName": "Global",
        "sexId": 3,
        "sexName": "Both",

```

```
    "ageId": 22,
    "ageName": "All ages",
    "metricId": 3,
    "metricName": "Rate",
    "year": 2021,
    "value": 5295.1,
    "upper": 6000.2,
    "lower": 4800.3,
    "populationGroupName": "All population"
  }
],
"total": 3200,
"limit": 100
},
"query": {
  "causeName": "Diabetes mellitus",
  "measureName": "Prevalence",
  "locationName": "Global",
  "sexName": "Both",
  "metricName": "Rate",
  "startYear": 2019,
  "endYear": 2021,
  "limit": 100
},
"message": "ok"
}
```

顶层字段说明：

字段	类型	说明
error	number	错误码，0 表示成功
data	object	GBD 查询结果，包含 rows、total 和 limit
query	object	服务端回显的规范化查询参数
message	string	人类可读消息，成功时通常为 ok

data.rows 字段说明：

字段	类型	说明
causeId	number	GBD cause ID
causeName	string	GBD cause / 疾病名称
measureId	number	GBD measure ID
measureName	string	指标名称
locationId	number	地区 ID
locationName	string	地区名称
sexId	number	性别 ID
sexName	string	性别名称
ageId	number	年龄组 ID
ageName	string	年龄组名称
metricId	number	计量方式 ID
metricName	string	计量方式名称
year	number	数据年份
value	number null	估计值
upper	number null	置信区间上界
lower	number null	置信区间下界
populationGroupName	string	人群分组名称（如底层数据存在）

错误响应（缺少搜索条件）：

```
{
  "statusCode": 400,
  "message": "At least one search condition is required: causeName, measureName, locationName, sexName, metricName, year, startYear, or endYear",
  "error": "Bad Request"
}
```

注意事项：

- 本接口只查询 GBD 结构化数据，不混合返回 EPI 文献抽取数据。
- total 表示底层命中总数，可能大于本次返回的 rows.length。
- 如果需要完整拉取大量命中结果，请拆分年份、地区、指标等过滤条件分批请求。
- limit 超过 3000 会被参数校验拒绝。

12. 靶点同源数据查询

查询同源关系数据，支持按一个或多个靶点 gene symbol 获取对应同源基因列表。接口为同步分页查询，适合在外部系统中为靶点详情页、物种同源关系展示或后续分析流程提供结构化数据。

接口类型： 同步（实时返回）
Content-Type: application/json
典型耗时： 200 ms - 2 秒（受命中量和分页位置影响）
响应超时： 30 秒
并发限制： 20 个/秒
数据源： OpenTargets homologues 本地镜像
依赖外部 LLM： 否

12.1 查询靶点同源数据

```
POST /api/homologues/search
Content-Type: application/json
```

12.2 请求参数

字段	类型	必填	默认值	说明
targets	string[]	是	-	靶点 gene symbol 列表，最多 50 个；服务端会按大小写不敏感方式匹配
pageNum	integer	否	1	页码，从 1 开始
pageSize	integer	否	10	每页返回条数，范围 1-100

12.3 请求示例

```
{
  "targets": ["CDK2", "EGFR"],
  "pageNum": 1,
  "pageSize": 10
}
```

curl 示例：

```
curl -X POST "https://search-alpha.metastar-health.com/prod-api/api/homologues/search" \
-H "Content-Type: application/json" \
-H "x-app-key: your_app_key" \
-H "x-app-secret: your_app_secret" \
-d '{
  "targets": ["CDK2", "EGFR"],
  "pageNum": 1,
  "pageSize": 10
}'
```

JavaScript 示例：

```
const response = await fetch(
  "https://search-alpha.metastar-health.com/prod-api/api/homologues/search",
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
    body: JSON.stringify({
```

```
        targets: ["CDK2", "EGFR"],
        pageNum: 1,
        pageSize: 10,
    }},
},
);

const result = await response.json();
```

Python 示例:

```
import requests

url = "https://search-alpha.metastar-health.com/prod-api/api/homologues/search"
headers = {
    "Content-Type": "application/json",
    "x-app-key": "your_app_key",
    "x-app-secret": "your_app_secret",
}
payload = {
    "targets": ["CDK2", "EGFR"],
    "pageNum": 1,
    "pageSize": 10,
}

response = requests.post(url, headers=headers, json=payload)
result = response.json()
```

12.4 返回参数

参数	类型	说明
<code>error</code>	number	错误码，成功为 0
<code>data.list</code>	object[]	当前页同源关系记录
<code>data.total</code>	number	命中总数
<code>data.pageNum</code>	integer	当前页码
<code>data.pageSize</code>	integer	当前每页条数
<code>data.totalPages</code>	integer	总页数
<code>data.hasNext</code>	boolean	是否还有下一页

`data.list[]` 字段说明:

参数	类型	说明
approvedSymbol	string	原始靶点 symbol
speciesName	string	同源基因所属物种
homologyType	string	同源关系类型，例如 ortholog_one2one、ortholog_one2many、ortholog_many2many、within_species_paralog、other_paralog
targetGeneSymbol	string	同源基因 symbol
queryPercentageIdentity	number	查询靶点相对同源序列的 identity 百分比
targetPercentageIdentity	number	同源基因相对查询序列的 identity 百分比
isHighConfidence	boolean null	高置信同源关系标记；部分 paralog 记录可能为 null

12.5 返回示例

```
{
  "error": 0,
  "data": {
    "list": [
      {
        "approvedSymbol": "CDK2",
        "speciesName": "Human",
        "homologyType": "other_paralog",
        "targetGeneSymbol": "CDK5",
        "queryPercentageIdentity": 58.0537,
        "targetPercentageIdentity": 59.2466,
        "isHighConfidence": null
      },
      {
        "approvedSymbol": "CDK2",
        "speciesName": "Mouse",
        "homologyType": "ortholog_one2one",
        "targetGeneSymbol": "Cdk2",
        "queryPercentageIdentity": 94.1176,
        "targetPercentageIdentity": 94.1176,
        "isHighConfidence": true
      }
    ],
    "total": 35,
    "pageNum": 1,
    "pageSize": 10,
    "totalPages": 4,
    "hasNext": true
  }
}
```

错误响应（缺少 targets）：

```
{
  "message": "targets is required",
  "error": 1001
}
```

12.6 注意事项

- `targets` 支持多个 symbol，多个 symbol 之间为 **OR** 关系；响应中的 `approvedSymbol` 可用于区分每条记录属于哪个查询靶点。
- 查询匹配大小写不敏感；例如 `c6orf141`、`C6ORF141` 都可命中原始 `approvedSymbol` 为 `C6orf141` 的记录。
- `approvedSymbol` 是 OpenTargets 原始 symbol，不是 Ensembl gene ID；接口不返回 `targetId` 或 `targetGeneId`。
- 极少数 symbol 命中量较大，请通过 `pageNum` + `pageSize` 翻页获取结果。

13. 专利临床前数据检索

接口类型：同步执行（`execute`）+ 异步任务（`submit` + 轮询）
Content-Type: `application/json`
典型耗时：1 - 15 秒
超时上限：30 秒
建议轮询间隔：5 - 10 秒（仅异步任务）
推荐并发：5 个
单次上限：50
结果保留：热数据 30 天；归档数据 90 天
上游限流/错误处理：无外部 QPM 预限流。

13.1 接口地址

项目	值
请求方式	<code>POST</code>
同步地址	<code>/api/v1/tools/preclinical_from_patent/execute</code>
异步提交地址	<code>/api/v1/tools/preclinical_from_patent/submit</code>
异步任务查询地址	<code>/api/v1/tools/tasks/:taskId</code>
Content-Type	<code>application/json</code>
接口标识	<code>preclinical_from_patent</code>
数据源	Elasticsearch
单次上限	50

13.2 请求参数

参数	类型	必填	说明
patentNumbers	string[]	否	专利号列表
classifications	string[]	否	专利分类号列表
targets	object[]	否	靶点实体
diseases	object[]	否	疾病实体
drugs	object[]	否	药物实体
companies	object[]	否	公司实体
limit	integer	否	返回数量，默认 50，不超过 50

13.3 请求示例

```
{
  "patentNumbers": [ "WO2024000001" ],
  "limit": 10
}
```

13.4 返回参数

参数	类型	说明
data.toolName	string	实际执行的接口标识
data.status	string	执行状态，成功为 ok
data.data	object	业务结果数据，包含 data 数组和 searchParams
data.total	number	可选，返回的展开后结果数，通常等于 data.data.data.length
data.truncated	boolean	可选，结果是否因超过大小限制被截断
data.originalBytes	number	可选，截断前结果大小，单位字节
error	number	错误码，成功为 0

data.data：专利临床前实验相关条目对象，其中 data.data.data 为展开后的实验结果数组，data.data.searchParams 为本次生效的检索参数。

异步提交接口返回 data.taskId 和 data.status；任务完成后，GET /api/v1/tools/tasks/:taskId 中的 data.result 与同步执行返回结构一致。

13.5 返回示例

```
{
  "data": {
    "toolName": "preclinical_from_patent",
    "status": "ok",
    "data": {
      "data": [
        {
          "patent_number": "WO2019033041",
          "target": [ "CDK2" ],

```

```
    "company": [ "Example Pharma" ],
    "experiment_id": "exp-1",
    "experiment_type": "in vivo"
  }
],
"searchParams": {
  "limit": 1
}
},
"total": 1
},
"error": 0
}
```

14. 专利核心库检索

接口类型： 同步执行（execute）+ 异步任务（submit + 轮询）

Content-Type: application/json

典型耗时： 5 - 30 秒

超时上限： 60 秒

建议轮询间隔： 5 - 10 秒（仅异步任务）

推荐并发： 5 个

单次上限： 50

结果保留： 热数据 30 天；归档数据 90 天

上游限流/错误处理： 不做智慧芽 QPM 预限流；智慧芽返回限流、超时或错误时，同步接口直接返回错误，异步任务标记为 failed。

14.1 接口地址

项目	值
请求方式	POST
同步地址	/api/v1/tools/zhy_api_synapse_patent_core_search/execute
异步提交地址	/api/v1/tools/zhy_api_synapse_patent_core_search/submit
异步任务查询地址	/api/v1/tools/tasks/:taskId
Content-Type	application/json
接口标识	zhy_api_synapse_patent_core_search
数据源	智慧芽 Synapse
单次上限	50

14.2 请求参数

参数	类型	必填	说明
keyword	string string[]	是	检索关键词，支持单个或多个关键词
searchType	string	否	检索类型，默认 keyword；可选 drug、target、disease、organization、company
limit	integer	否	返回数量，默认 20，不超过 50

14.3 请求示例

```
{
  "keyword": "EGFR",
  "searchType": "target"
}
```

14.4 返回参数

参数	类型	说明
data.toolName	string	实际执行的接口标识
data.status	string	执行状态，成功为 ok
data.data	object	业务结果数据，主要包含 patents、total
data.total	number	可选，返回的专利结果数，通常等于 data.data.patents.length
data.truncated	boolean	可选，结果是否因超过大小限制被截断
data.originalBytes	number	可选，截断前结果大小，单位字节
error	number	错误码，成功为 0

data.data：专利核心库检索结果对象，其中 data.data.patents 为专利结果数组。

异步提交接口返回 data.taskId 和 data.status；任务完成后，GET /api/v1/tools/tasks/:taskId 中的 data.result 与同步执行返回结构一致。

14.5 返回示例

```
{
  "data": {
    "toolName": "zhy_api_synapse_patent_core_search",
    "status": "ok",
    "data": {
      "patents": [
        {
          "patent_id": "patent-1",
          "title": "Example patent title",
          "patent_number": "WO2024000001",
          "current_assignee_names": ["Example Pharma"]
        }
      ],
      "total": 1
    }
  }
}
```

```
    },
    "total": 1
  },
  "error": 0
}
```

15. 临床试验注册检索

接口类型：同步执行（`execute`）+ 异步任务（`submit` + 轮询）

Content-Type: `application/json`

典型耗时：5 - 30 秒

超时上限：60 秒

建议轮询间隔：5 - 10 秒（仅异步任务）

推荐并发：5 个

单次上限：50

结果保留：热数据 30 天；归档数据 90 天

上游限流/错误处理：不做智慧芽 QPM 预限流；智慧芽返回限流、超时或错误时，同步接口直接返回错误，异步任务标记为 failed。

15.1 接口地址

项目	值
请求方式	<code>POST</code>
同步地址	<code>/api/v1/tools/zh_y_api_synapse_clinical_trial_search/execute</code>
异步提交地址	<code>/api/v1/tools/zh_y_api_synapse_clinical_trial_search/submit</code>
异步任务查询地址	<code>/api/v1/tools/tasks/:taskId</code>
Content-Type	<code>application/json</code>
接口标识	<code>zh_y_api_synapse_clinical_trial_search</code>
数据源	智慧芽 Synapse
单次上限	50

15.2 请求参数

参数	类型	必填	说明
<code>keyword</code>	string string[]	是	检索关键词，支持单个或多个关键词
<code>searchType</code>	string	否	检索类型，默认 <code>keyword</code> ；可选 <code>drug</code> 、 <code>target</code> 、 <code>disease</code> 、 <code>organization</code> 、 <code>company</code>
<code>limit</code>	integer	否	返回数量，默认 20，不超过 50

15.3 请求示例

```
{
  "keyword": "Osimertinib",
  "searchType": "drug",
  "limit": 10
}
```

15.4 返回参数

参数	类型	说明
data.toolName	string	实际执行的接口标识
data.status	string	执行状态，成功为 ok
data.data	object	业务结果数据，主要包含 trials、total
data.total	number	可选，返回的临床试验注册结果数，通常等于 data.data.trials.length
data.truncated	boolean	可选，结果是否因超过大小限制被截断
data.originalBytes	number	可选，截断前结果大小，单位字节
error	number	错误码，成功为 0

data.data：临床试验注册检索结果对象，其中 data.data.trials 为临床试验注册结果数组。

异步提交接口返回 data.taskId 和 data.status；任务完成后，GET /api/v1/tools/tasks/:taskId 中的 data.result 与同步执行返回结构一致。

15.5 返回示例

```
{
  "data": {
    "toolName": "zhy_api_synapse_clinical_trial_search",
    "status": "ok",
    "data": {
      "trials": [
        {
          "clinical_trial_id": "trial-1",
          "register_number": "NCT00000001",
          "brief_title": "Example clinical trial"
        }
      ],
      "total": 1
    },
    "total": 1
  },
  "error": 0
}
```

16. 临床试验结果检索

- 接口类型：同步执行（execute）+ 异步任务（submit + 轮询）
- Content-Type: application/json
- 典型耗时：5 - 60 秒
- 超时上限：90 秒

建议轮询间隔：5 - 10 秒（仅异步任务）
推荐并发：5 个
单次上限：20
结果保留：热数据 30 天；归档数据 90 天
上游限流/错误处理：不做智慧芽 QPM 预限流；智慧芽返回限流、超时或错误时，同步接口直接返回错误，异步任务标记为 failed。

16.1 接口地址

项目	值
请求方式	POST
同步地址	/api/v1/tools/zhy_api_synapse_ct_result_search/execute
异步提交地址	/api/v1/tools/zhy_api_synapse_ct_result_search/submit
异步任务查询地址	/api/v1/tools/tasks/:taskId
Content-Type	application/json
接口标识	zhy_api_synapse_ct_result_search
数据源	智慧芽 Synapse
单次上限	20

16.2 请求参数

参数	类型	必填	说明
keyword	string string[]	是	检索关键词，支持单个或多个关键词
searchType	string	否	检索类型，默认 keyword；可选 drug、target、disease、organization、company
limit	integer	否	返回数量及详情展开数量，默认 20，不超过 20

16.3 请求示例

```
{
  "keyword": "Osimertinib",
  "searchType": "drug",
  "limit": 5
}
```

16.4 返回参数

参数	类型	说明
<code>data.toolName</code>	string	实际执行的接口标识
<code>data.status</code>	string	执行状态，成功为 <code>ok</code>
<code>data.data</code>	object	业务结果数据，主要包含 <code>results</code> 、 <code>total</code>
<code>data.total</code>	number	可选，返回的临床试验结果数，通常等于 <code>data.data.results.length</code>
<code>data.truncated</code>	boolean	可选，结果是否因超过大小限制被截断
<code>data.originalBytes</code>	number	可选，截断前结果大小，单位字节
<code>error</code>	number	错误码，成功为 0

`data.data`：临床试验结果对象，其中 `data.data.results` 为临床试验结果数组。

异步提交接口返回 `data.taskId` 和 `data.status`；任务完成后，`GET /api/v1/tools/tasks/:taskId` 中的 `data.result` 与同步执行返回结构一致。

16.5 返回示例

```
{
  "data": {
    "toolName": "zhy_api_synapse_ct_result_search",
    "status": "ok",
    "data": {
      "results": [
        {
          "result_id": "result-1",
          "title": "Example clinical trial result",
          "phase": "Phase 2"
        }
      ],
      "total": 1
    },
    "total": 1
  },
  "error": 0
}
```

17. 药物交易检索

接口类型：同步执行（`execute`）+ 异步任务（`submit` + 轮询）

Content-Type: `application/json`

典型耗时：5 - 30 秒

超时上限：60 秒

建议轮询间隔：5 - 10 秒（仅异步任务）

推荐并发：5 个

单次上限：50

结果保留：热数据 30 天；归档数据 90 天

上游限流/错误处理：不做智慧芽 QPM 预限流；智慧芽返回限流、超时或错误时，同步接口直接返回错误，异步任务标记为 failed。

17.1 接口地址

项目	值
请求方式	POST
同步地址	/api/v1/tools/zhy_api_synapse_drug_deal_search/execute
异步提交地址	/api/v1/tools/zhy_api_synapse_drug_deal_search/submit
异步任务查询地址	/api/v1/tools/tasks/:taskId
Content-Type	application/json
接口标识	zhy_api_synapse_drug_deal_search
数据源	智慧芽 Synapse
单次上限	50

17.2 请求参数

参数	类型	必填	说明
keyword	string string[]	是	检索关键词，支持单个或多个关键词
searchType	string	否	检索类型，默认 keyword；可选 drug、target、disease、organization、company
limit	integer	否	返回数量，默认 20，不超过 50

17.3 请求示例

```
{
  "keyword": "EGFR",
  "searchType": "target"
}
```

17.4 返回参数

参数	类型	说明
data.toolName	string	实际执行的接口标识
data.status	string	执行状态，成功为 ok
data.data	object	业务结果数据，主要包含 deals、total
data.total	number	可选，返回的药物交易结果数，通常等于 data.data.deals.length
data.truncated	boolean	可选，结果是否因超过大小限制被截断
data.originalBytes	number	可选，截断前结果大小，单位字节
error	number	错误码，成功为 0

data.data：药物授权、并购、合作等交易结果对象，其中 data.data.deals 为交易结果数组。

异步提交接口返回 `data.taskId` 和 `data.status`；任务完成后，`GET /api/v1/tools/tasks/:taskId` 中的 `data.result` 与同步执行返回结构一致。

17.5 返回示例

```
{
  "data": {
    "toolName": "zhy_api_synapse_drug_deal_search",
    "status": "ok",
    "data": {
      "deals": [
        {
          "deal_id": "deal-1",
          "title": "Example collaboration agreement",
          "deal_status": "Completed"
        }
      ],
      "total": 1
    },
    "total": 1
  },
  "error": 0
}
```

18. 靶点竞争分析

接口类型：同步执行（`execute`）+ 异步任务（`submit` + 轮询）

Content-Type：`application/json`

典型耗时：5 - 60 秒

超时上限：60 秒

建议轮询间隔：5 - 10 秒（仅异步任务）

推荐并发：5 个

单次上限：50 个靶点关键词

结果保留：热数据 30 天；归档数据 90 天

上游限流/错误处理：不做智慧芽 QPM 预限流；智慧芽返回限流、超时或错误时，同步接口直接返回错误，异步任务标记为 failed。

18.1 接口地址

项目	值
请求方式	POST
同步地址	/api/v1/tools/zhy_api_synapse_target_analysis/execute
异步提交地址	/api/v1/tools/zhy_api_synapse_target_analysis/submit
异步任务查询地址	/api/v1/tools/tasks/:taskId
Content-Type	application/json
接口标识	zhy_api_synapse_target_analysis
数据源	智慧芽 Synapse
单次上限	50 个靶点关键词

18.2 请求参数

说明：本接口没有 `limit` 参数。单次上限 50 个靶点关键词 只限制 `keyword` 数组长度，不限制返回的 `competition` 数量。接口会返回智慧芽 Synapse 对解析后靶点的完整竞争格局结果，因此 `data.total` 或 `data.data.competition.length` 可能大于 50；超大响应仍受统一结果大小限制保护。

参数	类型	必填	说明
<code>keyword</code>	<code>string string[]</code>	是	靶点关键词，支持单个或多个；数组长度不超过 50
<code>searchType</code>	<code>string</code>	否	默认 <code>target</code> 或 <code>keyword</code>
<code>targetAliases</code>	<code>string[][]</code>	否	辅助靶点 ID 解析

18.3 请求示例

```
{
  "keyword": "EGFR",
  "searchType": "target"
}
```

18.4 返回参数

参数	类型	说明
<code>data.toolName</code>	<code>string</code>	实际执行的接口标识
<code>data.status</code>	<code>string</code>	执行状态，成功为 <code>ok</code>
<code>data.data</code>	<code>object</code>	靶点竞争分析结果，主要包含 <code>competition</code> 、 <code>targetInfo</code> 、 <code>targetNames</code> 、 <code>summary</code> 等字段
<code>data.total</code>	<code>number</code>	可选，返回的竞争格局结果数，通常等于 <code>data.data.competition.length</code>
<code>data.truncated</code>	<code>boolean</code>	可选，结果是否因超过大小限制被截断
<code>data.originalBytes</code>	<code>number</code>	可选，截断前结果大小，单位字节
<code>error</code>	<code>number</code>	错误码，成功为 0

`data.data`：靶点竞争/分析结果。该接口不按返回条数分页或截断，`competition` 数量取决于智慧芽 Synapse 返回的该靶点竞争格局数据。

异步提交接口返回 `data.taskId` 和 `data.status`；任务完成后，`GET /api/v1/tools/tasks/:taskId` 中的 `data.result` 与同步执行返回结构一致。

18.5 返回示例

```
{
  "data": {
    "toolName": "zhy_api_synapse_target_analysis",
    "status": "ok",
    "data": {
      "competition": [
        {
          "organization_name": "Example Pharma",
          "target_name": "EGFR",

```

```
        "total_drug_count": 3,
        "phase2_count": 1,
        "approved_count": 1
    }
],
"targetInfo": [
    {
        "targetName": "EGFR",
        "competitionCount": 1
    }
],
"targetNames": ["EGFR"],
"total": 1,
"alreadyProcessed": true,
"summary": {
    "totalTargets": 1,
    "analyzedTargets": 1,
    "totalCompetitors": 1
}
},
"total": 1
},
"error": 0
}
```

19. 医药行业资讯检索

接口类型：同步执行（`execute`）+ 异步任务（`submit` + 轮询）

Content-Type: `application/json`

典型耗时：5 - 30 秒

超时上限：60 秒

建议轮询间隔：5 - 10 秒（仅异步任务）

推荐并发：5 个

单次上限：50

结果保留：热数据 30 天；归档数据 90 天

上游限流/错误处理：不做智慧芽 QPM 预限流；智慧芽返回限流、超时或错误时，同步接口直接返回错误，异步任务标记为 failed。

19.1 接口地址

项目	值
请求方式	POST
同步地址	/api/v1/tools/zhy_api_synapse_news_search/execute
异步提交地址	/api/v1/tools/zhy_api_synapse_news_search/submit
异步任务查询地址	/api/v1/tools/tasks/:taskId
Content-Type	application/json
接口标识	zhy_api_synapse_news_search
数据源	智慧芽 Synapse
单次上限	50

19.2 请求参数

参数	类型	必填	说明
keyword	string string[]	是	检索关键词，支持单个或多个关键词
searchType	string	否	检索类型，默认 keyword；可选 drug、target、disease、organization、company
targetAliases	string[][]	否	searchType=target 时辅助解析靶点别名
targetCondition	string	否	靶点组合条件，如 AND / OR
limit	integer	否	返回数量，默认 20，不超过 50

19.3 请求示例

```
{
  "keyword": "EGFR",
  "searchType": "target"
}
```

19.4 返回参数

参数	类型	说明
<code>data.toolName</code>	string	实际执行的接口标识
<code>data.status</code>	string	执行状态，成功为 <code>ok</code>
<code>data.data</code>	object	业务结果数据，主要包含 <code>news</code> 、 <code>total</code> 、 <code>searchType</code> 、 <code>keyword</code>
<code>data.total</code>	number	可选，返回的资讯结果数，通常等于 <code>data.data.news.length</code>
<code>data.truncated</code>	boolean	可选，结果是否因超过大小限制被截断
<code>data.originalBytes</code>	number	可选，截断前结果大小，单位字节
<code>error</code>	number	错误码，成功为 0

`data.data`：医药行业资讯结果对象，其中 `data.data.news` 为资讯结果数组。

异步提交接口返回 `data.taskId` 和 `data.status`；任务完成后，`GET /api/v1/tools/tasks/:taskId` 中的 `data.result` 与同步执行返回结构一致。

19.5 返回示例

```
{
  "data": {
    "toolName": "zhy_api_synapse_news_search",
    "status": "ok",
    "data": {
      "news": [
        {
          "news_id": "news-1",
          "title": "Example pharma news",
          "publish_date": "2026-05-01",
          "source_url": "https://example.com/news"
        }
      ],
      "total": 1,
      "searchType": "target",
      "keyword": "EGFR"
    },
    "total": 1
  },
  "error": 0
}
```

20. 医学文献检索

接口类型：同步执行（`execute`）+ 异步任务（`submit` + 轮询）

Content-Type: `application/json`

典型耗时：5 - 30 秒

超时上限：60 秒

建议轮询间隔：5 - 10 秒（仅异步任务）

推荐并发：5 个

单次上限：50

结果保留：热数据 30 天；归档数据 90 天

上游限流/错误处理：不做平台侧 provider QPM 等待；上游返回限流、超时或错误时，同步接口直接返回错误，异步任务

标记为 failed。

20.1 接口地址

项目	值
请求方式	POST
同步地址	/api/v1/tools/search_pubmed/execute
异步提交地址	/api/v1/tools/search_pubmed/submit
异步任务查询地址	/api/v1/tools/tasks/:taskId
Content-Type	application/json
接口标识	search_pubmed
数据源	PubMed API
单次上限	50

20.2 请求参数

参数	类型	必填	说明
query	string	是	PubMed 检索式或自然语言关键词
maxArticles	number	否	最大文章数，默认 20，不超过 50

20.3 请求示例

```
{
  "query": "EGFR lung cancer resistance",
  "maxArticles": 20
}
```

20.4 返回参数

参数	类型	说明
data.toolName	string	实际执行的接口标识
data.status	string	执行状态，成功为 ok
data.data	object	业务结果数据，主要包含 articles
data.total	number	可选，返回的 PubMed 文献结果数，通常等于 data.data.articles.length
data.truncated	boolean	可选，结果是否因超过大小限制被截断
data.originalBytes	number	可选，截断前结果大小，单位字节
error	number	错误码，成功为 0

data.data：PubMed 文献结果对象，其中 data.data.articles 为文献记录数组，通常包含 PMID、标题、摘要、期刊、发表日期、作者、DOI 和链接等字段。

异步提交接口返回 `data.taskId` 和 `data.status`；任务完成后，`GET /api/v1/tools/tasks/:taskId` 中的 `data.result` 与同步执行返回结构一致。

20.5 返回示例

```
{
  "data": {
    "toolName": "search_pubmed",
    "status": "ok",
    "data": {
      "articles": [
        {
          "PMID": "00000000",
          "title": "Example PubMed article",
          "abstract": "Example abstract",
          "journal": "Example Journal",
          "published_date": "2026"
        }
      ]
    },
    "total": 1
  },
  "error": 0
}
```

21. 企业动态检索

- 接口类型：同步执行（`execute`）+ 异步任务（`submit` + 轮询）
- Content-Type: `application/json`
- 典型耗时：1 - 15 秒
- 超时上限：30 秒
- 建议轮询间隔：5 - 10 秒（仅异步任务）
- 推荐并发：5 个
- 单次上限：50
- 结果保留：热数据 30 天；归档数据 90 天
- 上游限流/错误处理：无外部 QPM 预限流。

21.1 接口地址

项目	值
请求方式	POST
同步地址	/api/v1/tools/search_medwatch/execute
异步提交地址	/api/v1/tools/search_medwatch/submit
异步任务查询地址	/api/v1/tools/tasks/:taskId
Content-Type	application/json
接口标识	search_medwatch
数据源	Elasticsearch / 微信文章库 / 公司动态数据
单次上限	50

21.2 请求参数

参数	类型	必填	说明
targets	object[]	否	靶点实体，建议使用自动补全接口返回的对象
diseases	object[]	否	疾病实体
drugs	object[]	否	药物实体
companies	object[]	否	公司实体
topics	string[]	否	主题过滤
limit	integer	否	返回数量，默认 50，不超过 50

21.3 请求示例

```
{
  "companies": [
    {
      "en": "AstraZeneca"
    }
  ],
  "limit": 10
}
```

21.4 返回参数

参数	类型	说明
data.toolName	string	实际执行的接口标识
data.status	string	执行状态，成功为 ok
data.data	array	企业动态文章记录数组
data.total	number	可选，返回的企业动态结果数，通常等于 data.data.length
data.truncated	boolean	可选，结果是否因超过大小限制被截断
data.originalBytes	number	可选，截断前结果大小，单位字节
error	number	错误码，成功为 0

data.data：企业动态文章记录，通常包含标题、摘要、发布时间、链接、匹配实体和来源信息。

异步提交接口返回 data.taskId 和 data.status；任务完成后，GET /api/v1/tools/tasks/:taskId 中的 data.result 与同步执行返回结构一致。

21.5 返回示例

```
{
  "data": {
    "toolName": "search_medwatch",
    "status": "ok",
    "data": [],
    "total": 0
  },
  "error": 0
}
```

22. 临床试验全域检索

接口类型：同步执行（execute）+ 异步任务（submit + 轮询）

Content-Type：application/json

典型耗时：1 - 15 秒

超时上限：30 秒

建议轮询间隔：5 - 10 秒（仅异步任务）

推荐并发：5 个

单次上限：200

结果保留：热数据 30 天；归档数据 90 天

上游限流/错误处理：无外部 QPM 预限流。

22.1 接口地址

项目	值
请求方式	POST
同步地址	/api/v1/tools/search_trial/execute
异步提交地址	/api/v1/tools/search_trial/submit
异步任务查询地址	/api/v1/tools/tasks/:taskId
Content-Type	application/json
接口标识	search_trial
数据源	Elasticsearch
单次上限	200

22.2 请求参数

参数	类型	必填	说明
targets	object[]	否	靶点实体，建议使用自动补全接口返回的对象
diseases	object[]	否	疾病实体
drugs	object[]	否	药物实体
companies	object[]	否	公司实体
limit	integer	否	返回数量，默认 200，不超过 200

22.3 请求示例

```
{
  "drugs": [
    {
      "en": "Osimertinib"
    }
  ],
  "limit": 10
}
```

22.4 返回参数

参数	类型	说明
data.toolName	string	实际执行的接口标识
data.status	string	执行状态，成功为 ok
data.data	array	临床试验记录数组
data.total	number	可选，返回的临床试验结果数，通常等于 data.data.length
data.truncated	boolean	可选，结果是否因超过大小限制被截断
data.originalBytes	number	可选，截断前结果大小，单位字节
error	number	错误码，成功为 0

data.data：临床试验记录数组，通常包含 trial id、标题、适应症、药物、靶点、阶段、状态和申办方。

异步提交接口返回 data.taskId 和 data.status；任务完成后，GET /api/v1/tools/tasks/:taskId 中的 data.result 与同步执行返回结构一致。

22.5 返回示例

```
{
  "data": {
    "toolName": "search_trial",
    "status": "ok",
    "data": [],
    "total": 0
  },
  "error": 0
}
```

23. 专利综合检索

- 接口类型：同步执行 (execute) + 异步任务 (submit + 轮询)
- Content-Type: application/json
- 典型耗时：1 - 15 秒
- 超时上限：30 秒
- 建议轮询间隔：5 - 10 秒（仅异步任务）
- 推荐并发：5 个
- 单次上限：200

结果保留：热数据 30 天；归档数据 90 天
上游限流/错误处理：无外部 QPM 预限流。

23.1 接口地址

项目	值
请求方式	POST
同步地址	/api/v1/tools/search_patent/execute
异步提交地址	/api/v1/tools/search_patent/submit
异步任务查询地址	/api/v1/tools/tasks/:taskId
Content-Type	application/json
接口标识	search_patent
数据源	Elasticsearch
单次上限	200

23.2 请求参数

参数	类型	必填	说明
targets	object[]	否	靶点实体，建议使用自动补全接口返回的对象
diseases	object[]	否	疾病实体
drugs	object[]	否	药物实体
companies	object[]	否	公司实体
limit	integer	否	返回数量，默认 200，不超过 200

23.3 请求示例

```
{
  "targets": [
    {
      "name": "EGFR"
    }
  ],
  "limit": 10
}
```

23.4 返回参数

参数	类型	说明
<code>data.toolName</code>	string	实际执行的接口标识
<code>data.status</code>	string	执行状态，成功为 <code>ok</code>
<code>data.data</code>	array	专利记录数组
<code>data.total</code>	number	可选，返回的专利结果数，通常等于 <code>data.data.length</code>
<code>data.truncated</code>	boolean	可选，结果是否因超过大小限制被截断
<code>data.originalBytes</code>	number	可选，截断前结果大小，单位字节
<code>error</code>	number	错误码，成功为 0

`data.data`：专利记录数组，通常包含专利号、标题、摘要、申请人、公开日、法律状态和实体命中。

异步提交接口返回 `data.taskId` 和 `data.status`；任务完成后，`GET /api/v1/tools/tasks/:taskId` 中的 `data.result` 与同步执行返回结构一致。

23.5 返回示例

```
{
  "data": {
    "toolName": "search_patent",
    "status": "ok",
    "data": [],
    "total": 0
  },
  "error": 0
}
```

24. 实时网页检索

接口类型：同步执行（`execute`）+ 异步任务（`submit` + 轮询）

Content-Type: `application/json`

典型耗时：5 - 30 秒

超时上限：60 秒

建议轮询间隔：5 - 10 秒（仅异步任务）

推荐并发：5 个

单次上限：50

结果保留：热数据 30 天；归档数据 90 天

上游限流/错误处理：不做平台侧 provider QPM 等待；上游返回限流、超时或错误时，同步接口直接返回错误，异步任务标记为 failed。

24.1 接口地址

项目	值
请求方式	POST
同步地址	/api/v1/tools/search_engine/execute
异步提交地址	/api/v1/tools/search_engine/submit
异步任务查询地址	/api/v1/tools/tasks/:taskId
Content-Type	application/json
接口标识	search_engine
数据源	SerpAPI / Serper / Mita
单次上限	50

24.2 请求参数

参数	类型	必填	说明
query	string	是	搜索问题或关键词
engines	string[]	否	可选 google、bing、baidu、mita；最多 4 个且不可重复；不传默认 Google
maxResults	integer	否	每个搜索引擎的返回数量，默认 10，不超过 50

24.3 请求示例

```
{
  "query": "EGFR exon 20 inhibitor latest clinical trial",
  "engines": ["google", "bing"],
  "maxResults": 10
}
```

24.4 返回参数

参数	类型	说明
data.toolName	string	实际执行的接口标识
data.status	string	执行状态，成功为 ok
data.data	object	业务结果数据，包含 query 和 results
data.total	number	可选，返回的搜索引擎结果组数量，通常等于 data.data.results.length
data.truncated	boolean	可选，结果是否因超过大小限制被截断
data.originalBytes	number	可选，截断前结果大小，单位字节
error	number	错误码，成功为 0

data.data：网页检索结果对象，其中 data.data.results 为按搜索引擎分组的结果数组；每个元素包含 engine 和该引擎返回的原始 data。maxResults 控制每个引擎内部返回条数，不是 results 分组数量。

异步提交接口返回 `data.taskId` 和 `data.status`；任务完成后，GET `/api/v1/tools/tasks/:taskId` 中的 `data.result` 与同步执行返回结构一致。

24.5 返回示例

```
{
  "data": {
    "toolName": "search_engine",
    "status": "ok",
    "data": {
      "query": "EGFR exon 20 inhibitor latest clinical trial",
      "results": [
        {
          "engine": "google",
          "data": {
            "organic": [
              {
                "title": "Example search result",
                "link": "https://example.com/article",
                "snippet": "Example snippet"
              }
            ]
          }
        }
      ]
    },
    "total": 1
  },
  "error": 0
}
```

25. 靶点综述文献检索

接口类型：同步执行（`execute`）+ 异步任务（`submit` + 轮询）

Content-Type: `application/json`

典型耗时：1 - 15 秒

超时上限：30 秒

建议轮询间隔：5 - 10 秒（仅异步任务）

推荐并发：5 个

单次上限：200

结果保留：热数据 30 天；归档数据 90 天

上游限流/错误处理：无外部 QPM 预限流。

25.1 接口地址

项目	值
请求方式	POST
同步地址	/api/v1/tools/search_review_combine/execute
异步提交地址	/api/v1/tools/search_review_combine/submit
异步任务查询地址	/api/v1/tools/tasks/:taskId
Content-Type	application/json
接口标识	search_review_combine
数据源	Elasticsearch
单次上限	200

25.2 请求参数

参数	类型	必填	说明
targets	object[]	否	靶点实体，建议使用自动补全接口返回的对象
diseases	object[]	否	疾病实体
topics	string[]	否	主题过滤
limit	integer	否	返回数量，默认 200，不超过 200

25.3 请求示例

```
{
  "targets": [
    {
      "name": "EGFR"
    }
  ],
  "limit": 10
}
```

25.4 返回参数

参数	类型	说明
data.toolName	string	实际执行的接口标识
data.status	string	执行状态，成功为 ok
data.data	array	靶点综述文献结构化记录数组
data.total	number	可选，返回的综述文献结果数，通常等于 data.data.length
data.truncated	boolean	可选，结果是否因超过大小限制被截断
data.originalBytes	number	可选，截断前结果大小，单位字节
error	number	错误码，成功为 0

`data.data`：靶点综述文献结构化记录，通常包含题名、摘要、期刊、年份、主题和实体命中。

异步提交接口返回 `data.taskId` 和 `data.status`；任务完成后，GET `/api/v1/tools/tasks/:taskId` 中的 `data.result` 与同步执行返回结构一致。

25.5 返回示例

```
{
  "data": {
    "toolName": "search_review_combine",
    "status": "ok",
    "data": [],
    "total": 0
  },
  "error": 0
}
```

26. 会议洞察

- 接口类型：同步执行（`execute`）+ 异步任务（`submit` + 轮询）
- Content-Type: `application/json`
- 典型耗时：1 - 15 秒
- 超时上限：30 秒
- 建议轮询间隔：5 - 10 秒（仅异步任务）
- 推荐并发：5 个
- 单次上限：200
- 结果保留：热数据 30 天；归档数据 90 天
- 上游限流/错误处理：无外部 QPM 预限流。

26.1 接口地址

项目	值
请求方式	<code>POST</code>
同步地址	<code>/api/v1/tools/search_conference_posters/execute</code>
异步提交地址	<code>/api/v1/tools/search_conference_posters/submit</code>
异步任务查询地址	<code>/api/v1/tools/tasks/:taskId</code>
Content-Type	<code>application/json</code>
接口标识	<code>search_conference_posters</code>
数据源	Elasticsearch
单次上限	200

26.2 请求参数

参数	类型	必填	说明
<code>targets</code>	<code>object[]</code>	否	靶点实体，建议使用自动补全接口返回的对象
<code>diseases</code>	<code>object[]</code>	否	疾病实体
<code>drugs</code>	<code>object[]</code>	否	药物实体
<code>companies</code>	<code>object[]</code>	否	公司实体
<code>limit</code>	<code>integer</code>	否	返回数量，默认 200，不超过 200

26.3 请求示例

```
{
  "targets": [
    {
      "name": "EGFR"
    }
  ],
  "limit": 10
}
```

26.4 返回参数

参数	类型	说明
<code>data.toolName</code>	<code>string</code>	实际执行的接口标识
<code>data.status</code>	<code>string</code>	执行状态，成功为 <code>ok</code>
<code>data.data</code>	<code>array</code>	会议 Poster 记录数组
<code>data.total</code>	<code>number</code>	可选，返回的会议 Poster 结果数，通常等于 <code>data.data.length</code>
<code>data.truncated</code>	<code>boolean</code>	可选，结果是否因超过大小限制被截断
<code>data.originalBytes</code>	<code>number</code>	可选，截断前结果大小，单位字节
<code>error</code>	<code>number</code>	错误码，成功为 0

`data.data`：会议 Poster 记录，通常包含题名、摘要、会议、年份、实体命中和来源信息。

异步提交接口返回 `data.taskId` 和 `data.status`；任务完成后，`GET /api/v1/tools/tasks/:taskId` 中的 `data.result` 与同步执行返回结构一致。

26.5 返回示例

```
{
  "data": {
    "toolName": "search_conference_posters",
    "status": "ok",
    "data": [],
    "total": 0
  },
  "error": 0
}
```

27. 公司演示文稿检索

接口类型：同步执行（execute）+ 异步任务（submit + 轮询）

Content-Type：application/json

典型耗时：1 - 15 秒

超时上限：30 秒

建议轮询间隔：5 - 10 秒（仅异步任务）

推荐并发：5 个

单次上限：200

结果保留：热数据 30 天；归档数据 90 天

上游限流/错误处理：无外部 QPM 预限流。

27.1 接口地址

项目	值
请求方式	POST
同步地址	/api/v1/tools/search_conference_presentations/execute
异步提交地址	/api/v1/tools/search_conference_presentations/submit
异步任务查询地址	/api/v1/tools/tasks/:taskId
Content-Type	application/json
接口标识	search_conference_presentations
数据源	Elasticsearch
单次上限	200

27.2 请求参数

参数	类型	必填	说明
targets	object[]	否	靶点实体，建议使用自动补全接口返回的对象
diseases	object[]	否	疾病实体
drugs	object[]	否	药物实体
companies	object[]	否	公司实体
limit	integer	否	返回数量，默认 200，不超过 200

27.3 请求示例

```
{
  "companies": [
    {
      "en": "AstraZeneca"
    }
  ],
  "limit": 10
}
```

27.4 返回参数

参数	类型	说明
<code>data.toolName</code>	string	实际执行的接口标识
<code>data.status</code>	string	执行状态，成功为 <code>ok</code>
<code>data.data</code>	array	公司演示文稿记录数组
<code>data.total</code>	number	可选，返回的公司演示文稿结果数，通常等于 <code>data.data.length</code>
<code>data.truncated</code>	boolean	可选，结果是否因超过大小限制被截断
<code>data.originalBytes</code>	number	可选，截断前结果大小，单位字节
<code>error</code>	number	错误码，成功为 0

`data.data`：公司演示文稿记录，通常包含标题、公司、发布日期、管线/靶点/适应症关联和来源信息。

异步提交接口返回 `data.taskId` 和 `data.status`；任务完成后，`GET /api/v1/tools/tasks/:taskId` 中的 `data.result` 与同步执行返回结构一致。

27.5 返回示例

```
{
  "data": {
    "toolName": "search_conference_presentations",
    "status": "ok",
    "data": [],
    "total": 0
  },
  "error": 0
}
```

28. 文献全文存在性查询

通过 PMID 或 DOI 查询平台是否存在 PubMed 全文 chunks。接口适合外部系统在做 RAG、证据追溯或全文阅读前先判断全文可用性。

服务端查询流程固定为两步：

- 1. 先在平台文献摘要库中精确匹配 `pmid` 或 `doi`。
- 2. 如果摘要库未命中，直接返回成功响应且 `data.chunks` 为空；如果命中，则使用解析出的 PMID 查询平台全文库的 chunks。

接口类型：同步（实时返回）

Content-Type: `application/json`

典型耗时：200 ms - 3 秒

响应超时：30 秒

并发限制：20 个/秒

数据源：平台文献摘要库、平台全文库

依赖外部 LLM：否

28.1 接口地址

```
POST /api/literature-fulltext/lookup
Content-Type: application/json
```

28.2 请求参数

参数	类型	必填	默认值	说明
pmid	string	否*	-	PubMed ID
doi	string	否*	-	DOI
limit	integer	否	100	返回全文 chunk 数量上限，范围 1-1000

* `pmid` 和 `doi` 至少提供一个。两者都提供时，服务端会在平台文献摘要库中按任一标识符命中后解析 PMID。

28.3 请求示例

```
{
  "pmid": "29021135",
  "limit": 100
}
```

或：

```
{
  "doi": "10.1038/nature12373",
  "limit": 100
}
```

curl 示例：

```
curl -X POST "https://search-alpha.metastar-health.com/prod-api/api/literature-fulltext/lookup" \
-H "Content-Type: application/json" \
-H "x-app-key: your_app_key" \
-H "x-app-secret: your_app_secret" \
-d '{
  "pmid": "29021135",
  "limit": 100
}'
```

28.4 返回参数

参数	类型	说明
<code>error</code>	number	错误码，成功为 0
<code>data.fulltextExists</code>	boolean	平台全文库是否返回至少一个 chunk
<code>data.article</code>	object	摘要库命中后的文献基础信息；未命中时不存在
<code>data.article.pmid</code>	string	解析出的 PMID
<code>data.article.doi</code>	string	解析出的 DOI
<code>data.article.title</code>	string	文献标题
<code>data.article.journalName</code>	string	期刊名称
<code>data.article.pubdate</code>	string	发表日期
<code>data.chunks</code>	object[]	全文 chunk 数组；摘要未命中或全文不存在时为空数组
<code>data.total</code>	number	本次返回的 chunk 数量
<code>data.limit</code>	integer	实际生效的 chunk 返回上限

`data.chunks[]` 字段说明：

参数	类型	说明
<code>chunkId</code>	string	chunk 文档 ID
<code>parentPmid</code>	string	chunk 所属 PMID
<code>chunkType</code>	string	chunk 类型，例如 <code>text</code> 、 <code>table</code> 、 <code>figure</code>
<code>sectionKey</code>	string	章节 key 或标题
<code>order</code>	number	chunk 在全文中的顺序
<code>content</code>	string	chunk 正文

28.5 返回示例

命中全文：

```
{
  "error": 0,
  "data": {
    "fulltextExists": true,
    "article": {
      "pmid": "29021135",
      "doi": "10.1038/nature12373",
      "title": "Example article title",
      "journalName": "Nature",
      "pubdate": "2017"
    },
    "chunks": [
      {
        "chunkId": "fulltext_chunk_001",
        "parentPmid": "29021135",
```

```
    "chunkType": "text",
    "sectionKey": "RESULTS",
    "order": 1,
    "content": "Full-text paragraph..."
  }
],
"total": 1,
"limit": 100
}
```

摘要库未命中或无全文 chunks:

```
{
  "error": 0,
  "data": {
    "fulltextExists": false,
    "chunks": [],
    "total": 0,
    "limit": 100
  }
}
```

28.6 注意事项

- 本接口只判断平台本地全文库中是否存在全文 chunks，不会调用 PubMed、PMC 或第三方接口实时下载全文。
- 当前全文库并非全量文献库，主要覆盖 IF > 5 的 Review 文献 和 IF > 10 的 Article 文献。
- 如果某篇文章返回 `fulltextExists=false` 或 `chunks=[]`，可能有两类原因：一是该文章确实无法获取全文；二是该文章尚未被扩充进当前全文库。请不要将未命中直接理解为文献不存在。
- `doi` 匹配依赖平台文献摘要库中保存的 DOI 字段；如果 DOI 未入库，即使同一 PMID 存在全文，也可能按未命中处理。已知 PMID 时优先传 `pmid`。
- `chunks` 可能较长，调用方可通过 `limit` 控制单次返回量；如需完整全文，建议先用默认值确认 `fulltextExists`。

29. 公司源列表查询

查询平台监控中的公司源列表，用于先获取公司标识（`companyKey`），再调用接口 30 拉取该公司下的内容。仅返回已启用（enabled）的公司。

接口类型：同步（实时返回）

Content-Type: `application/x-www-form-urlencoded` (query string)

典型耗时：100 - 500 ms

响应超时：10 秒

并发限制：20 个/秒

数据源：平台公司源配置库

29.1 查询公司源列表

```
GET /api/source-companies
```

请求参数：

参数名	类型	必填	说明
name	string	否	名称模糊匹配，对 displayName 或 companyKey 做大小写不敏感包含匹配（中英混合名称通用）；中文需按 UTF-8 URL 编码；最长 200 字符
pageNum	integer	否	页码，从 1 开始，默认 1
pageSize	integer	否	每页条数，默认 20，最大 100

JavaScript 示例：

```
const params = new URLSearchParams({
  name: "诺诚",
  pageNum: 1,
  pageSize: 20,
});

const response = await fetch(
  `https://search-alpha.metastar-health.com/prod-api/api/source-companies?${params.toString()}`,
  {
    headers: {
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
  },
);

const result = await response.json();
```

Python 示例：

```
import requests

url = "https://search-alpha.metastar-health.com/prod-api/api/source-companies"
headers = {
  "x-app-key": "your_app_key",
  "x-app-secret": "your_app_secret",
}
params = {
  "name": "诺诚",
  "pageNum": 1,
  "pageSize": 20,
}

response = requests.get(url, headers=headers, params=params)
result = response.json()
```

29.2 返回参数

参数	类型	说明
<code>error</code>	number	错误码，成功为 0
<code>data.list</code>	object[]	当前页公司列表
<code>data.total</code>	number	命中总数
<code>data.pageNum</code>	integer	当前页码
<code>data.pageSize</code>	integer	当前每页条数
<code>data.totalPages</code>	integer	总页数
<code>data.hasNext</code>	boolean	是否还有下一页

`data.list[]` 字段说明：

参数	类型	说明
<code>companyKey</code>	string	公司稳定标识，用于接口 30 的路径参数
<code>displayName</code>	string	公司展示名称（可能为中文）
<code>officialWebsite</code>	string	公司官网 URL，可能为空字符串

29.3 返回示例

```
{
  "error": 0,
  "data": {
    "list": [
      {
        "companyKey": "innocarepharma",
        "displayName": "诺诚健华",
        "officialWebsite": "https://www.innocarepharma.com"
      }
    ],
    "total": 11,
    "pageNum": 1,
    "pageSize": 20,
    "totalPages": 1,
    "hasNext": false
  }
}
```

29.4 注意事项

- 只返回已启用的公司；某公司被停用后不再出现在本列表，但其此前已采集的内容仍可通过接口 30 查询（内容不会被删除）。
- `companyKey` 是跨接口稳定引用；`displayName` 可能调整，请勿据其做持久化主键。

30. 公司源内容查询

按公司查询已采集的资讯（news）/ 演示文稿（presentation）列表条目，按 `sortAt`（发布时间 `publishedAt` 优先，缺失时回退到发现时间 `discoveredAt`）倒序排列。采用游标分页，适合按公司增量拉取。

接口类型： 同步（实时返回）
Content-Type: application/x-www-form-urlencoded （query string）
典型耗时： 200 ms - 2 秒
响应超时： 30 秒
并发限制： 20 个/秒
数据源： 平台公司源内容库

30.1 查询公司内容

GET /api/source-companies/{companyKey}/contents

路径参数：

参数名	类型	必填	说明
companyKey	string	是	公司标识（来自接口 29）；不存在时返回 EntityNotExists 错误

请求参数（query）：

参数名	类型	必填	说明
contentType	string	否	内容类型过滤，取值 news 或 presentation
startTime	string	否	ISO-8601 时间，作用于 sortAt 的闭区间起点，例如 2026-06-01T00:00:00Z
endTime	string	否	ISO-8601 时间，作用于 sortAt 的闭区间终点
cursor	string	否	翻页游标；首页不传，后续把上一页返回的 nextCursor 原样回传
limit	integer	否	单页条数，默认 20，最大 100

JavaScript 示例：

```
const params = new URLSearchParams({
  contentType: "news",
  startTime: "2026-06-01T00:00:00Z",
  limit: 20,
});

const response = await fetch(
  `https://search-alpha.metastar-health.com/prod-api/api/source-companies/innocarepharma/contents?${params.toString()}`,
  {
    headers: {
      "x-app-key": "your_app_key",
      "x-app-secret": "your_app_secret",
    },
  },
);

const result = await response.json();
```

Python 示例：

```
import requests
```

```
url = "https://search-alpha.metastar-health.com/prod-api/api/source-companies/innocarepharma/contents"
headers = {
    "x-app-key": "your_app_key",
    "x-app-secret": "your_app_secret",
}
params = {
    "contentType": "news",
    "startTime": "2026-06-01T00:00:00Z",
    "limit": 20,
}

response = requests.get(url, headers=headers, params=params)
result = response.json()
```

30.2 返回参数

参数	类型	说明
error	number	错误码，成功为 0
data.items	object[]	当前页内容条目
data.nextCursor	string null	下一页游标；为 null 表示已到末页
data.hasMore	boolean	是否还有下一页

data.items[] 字段说明：

参数	类型	说明
companyKey	string	所属公司标识
contentType	string	内容类型，news 或 presentation
title	string	标题
url	string	原文链接
publishedAt	string null	发布时间；源站日期无法解析时为 null
discoveredAt	string	首次发现时间（ISO-8601）
sortAt	string	排序时间轴 = publishedAt ?? discoveredAt，也是时间范围过滤的作用字段

30.3 返回示例

```
{
  "error": 0,
  "data": {
    "items": [
      {
        "companyKey": "innocarepharma",
        "contentType": "news",
        "title": "诺诚健华某新药获批临床",
        "url": "https://www.innocarepharma.com/news/activity/cn02...",
        "publishedAt": "2026-07-13",
        "discoveredAt": "2026-07-15T09:50:07Z",
        "sortAt": "2026-07-13T00:00:00.000Z"
      }
    ]
  }
}
```

```

    }
  ],
  "nextCursor": "MTc2Nz...",
  "hasMore": true
}
}

```

30.4 增量拉取模式

1. 首次拉取：记录本次拉取时间。
2. 后续拉取：把上次的拉取时间作为 `startTime` 传入，只取新增内容。
3. 单次结果分页：若 `hasMore` 为 `true`，带上返回的 `nextCursor` 继续请求下一页，直到 `hasMore` 为 `false`。

30.5 注意事项

- 时间范围作用于 `sortAt` 而非 `publishedAt`：因为 `publishedAt` 可能为 `null`，用 `sortAt` 才能与倒序/游标语义自洽，且不会漏掉无发布日期的条目。
- `cursor` 是不透明字符串，请原样回传，不要解析或拼接；游标格式错误、`startTime/endTime` 非法或 `startTime > endTime` 时返回 `InvalidParams` 错误。
- 已停用公司的历史内容仍可查询（内容不会被删除）。

完整集成示例

以下是使用 Python 完整演示异步任务工作流的示例：

```

import requests
import time

BASE_URL = "https://search-alpha.metastar-health.com/prod-api/api"
HEADERS = {
    "x-app-key": "your_app_key",
    "x-app-secret": "your_app_secret",
}

def submit_ocr(pdf_path: str) -> str:
    """提交 PDF 进行 OCR 处理，返回任务 ID。"""
    with open(pdf_path, "rb") as f:
        files = {"file": (pdf_path, f, "application/pdf")}
        resp = requests.post(f"{BASE_URL}/ocr/submit", headers=HEADERS, files=files)

    data = resp.json()
    if data["error"] != 0:
        raise Exception(f"提交失败: {data.get('message')}")

    return data["data"]["taskId"]

def poll_result(service: str, task_id: str, interval: int = 5, timeout: int = 600):
    """轮询任务结果，直到完成、失败或超时。"""
    elapsed = 0
    while elapsed < timeout:
        resp = requests.get(
            f"{BASE_URL}/{service}/result/{task_id}", headers=HEADERS
        )

```

```

    data = resp.json()["data"]
    status = data["status"]
    progress = data.get("progress", 0)

    print(f" [{elapsed}s] status={status}, progress={progress}%")

    if status == "completed":
        return data["result"]
    elif status == "failed":
        raise Exception(f"任务失败: {data.get('error')}")

    time.sleep(interval)
    elapsed += interval

    raise TimeoutError(f"任务 {task_id} 超时 ({timeout}s) ")

# 使用示例
task_id = submit_ocr("document.pdf")
print(f"任务已提交: {task_id}")

result = poll_result("ocr", task_id)
print(f"OCR 处理完成: {result.get('filename')} ({result.get('size', 0)} bytes)")
print(f"下载链接: {result.get('downloadUrl')}")

```

以下是使用 JavaScript/TypeScript 完整演示异步任务工作流的示例：

```

const BASE_URL = "https://search-alpha.metastar-health.com/prod-api/api";
const HEADERS = {
  "x-app-key": "your_app_key",
  "x-app-secret": "your_app_secret",
};

async function submitOcr(file) {
  const formData = new FormData();
  formData.append("file", file);

  const resp = await fetch(`${BASE_URL}/ocr/submit`, {
    method: "POST",
    headers: HEADERS,
    body: formData,
  });
  const data = await resp.json();

  if (data.error !== 0) {
    throw new Error(`提交失败: ${data.message}`);
  }
  return data.data.taskId;
}

async function pollResult(service, taskId, interval = 5000, timeout = 600000) {
  const start = Date.now();

  while (Date.now() - start < timeout) {
    const resp = await fetch(`${BASE_URL}/${service}/result/${taskId}`, {
      headers: HEADERS,
    });
    const { data } = await resp.json();
  }
}

```

```
console.log(`  status=${data.status}, progress=${data.progress}%`);

if (data.status === "completed") return data.result;
if (data.status === "failed") throw new Error(`任务失败: ${data.error}`);

await new Promise((r) => setTimeout(r, interval));
}

throw new Error(`任务 ${taskId} 超时`);
}

// 使用示例
const taskId = await submitOcr(pdfFile);
console.log(`任务已提交: ${taskId}`);

const result = await pollResult("ocr", taskId);
console.log(`OCR 处理完成: ${result.filename} (${result.size} bytes)`);
console.log(`下载链接: ${result.downloadUrl}`);
```